

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN - ĐIỆN TỬ**



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH
ỨNG DỤNG ROS2**

GVHD: PGS. TS. Phạm Hồng Liên

SVTH: Ngô Quốc Khánh MSSV: 22161269

Vũ Tấn Khoa MSSV: 17119086

HỌC KỲ 2 – NĂM HỌC 2025-2026

Thành phố Hồ Chí Minh, tháng 6 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH
ỨNG DỤNG ROS2

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ – VIỄN THÔNG

Sinh viên: Vũ Tấn Khoa

MSSV: 17119086

Ngô Quốc Khánh

MSSV: 22161269

Hướng dẫn: PSG. TS. Phạm Hồng Liên

Thành phố Hồ Chí Minh – 06/2026

LUẬN TỐT NGHIỆP

BẢN NHẬN XÉT KHÓA

hướng dẫn)

(Dành cho giảng viên

Đề tài: **THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH ỨNG DỤNG ROS2**

Sinh viên: + NGÔ QUỐC KHÁNH

MSSV: 22161269

+ VŨ TÂN KHOA

MSSV: 17119086

Hướng dẫn: **PGS. TS. Phạm Hồng Liên**

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Các tác giả đặt vấn đề cho khóa luận tốt nghiệp một cách rõ ràng, có mục tiêu cụ thể. Đề tài có nhiều tính mới, tính kế thừa và có khả năng ứng dụng rất cao vào thực tế. Các tác giả thiết kế mô hình robot tự hành có khả năng tự động di chuyển trong khu vực nhà hàng để thực hiện nhiệm vụ vận chuyển món ăn và đồ uống đến các bàn phục vụ. Ngoài ra đề tài còn phát triển nền tảng website nhằm hỗ trợ quản lý và theo dõi robot trong quá trình phục vụ. Vì vậy đề tài có khả năng ứng dụng rất cao vào thực tế.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp thực hiện đề tài hợp lý và tin cậy dựa trên cơ sở lý thuyết có phân tích và đánh giá từ đó đưa ra mô hình thiết kế phần cứng và các giải thuật phần mềm phù hợp. Hệ thống sử dụng Raspberry Pi 4 làm bộ xử lý trung tâm, Arduino Mega 2560 làm bộ điều khiển cấp thấp, cảm biến RPLidar A1 để thu dữ liệu môi trường và động cơ DC encoder JGB37-520 để di chuyển theo cấu hình bánh xe vi sai. Về phần mềm, đề tài sử dụng Ubuntu 22.04, ROS2 Humble, SLAM Toolbox, Navigation2 và RViz2 để triển khai các chức năng nhận thức không gian, định vị và điều hướng. Bên cạnh đó, hệ thống có thêm phần website phục vụ mục tiêu quản lý và tương tác với robot trong mô hình nhà hàng thông minh.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Kết quả thực hiện đề tài phù hợp với mục tiêu đề tài; thiết kế hợp lý; thi công phần cứng và phần mềm khá tối ưu. Việc kết hợp robot, ROS2, SLAM, Nav2 và website trong đề tài, vừa có tính kỹ thuật hệ thống nhúng, vừa có khả năng mở rộng thành ứng dụng thực tế.

Kết luận và đề xuất: Kết luận của khóa luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ. Khối lượng công việc của đề tài khá lớn, đòi hỏi sự nỗ lực rất cao của các tác giả.

4. Hình thức trình bày và bố cục báo cáo:

Văn phong của khóa luận nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt.

5. Kỹ năng chuyên nghiệp và tính sáng tạo:

Tác giả thể hiện các kỹ năng giao tiếp và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài khá tốt.

6. Tài liệu trích dẫn

Tài liệu trích dẫn trung thực và phù hợp với đề tài; trích dẫn theo đúng chỉ dẫn APA khá phong phú.

8. Đánh giá về sự trùng lặp của đề tài

Đề tài không trùng lặp với các đề tài khác.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Cần sửa một số lỗi chính tả.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần, thái độ học tập, nghiên cứu của sinh viên rất nghiêm túc, thường xuyên báo cáo tiến độ với giáo viên. Sinh viên luôn độc lập và tìm hiểu các tính mới của đề tài.

Kết luận : Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 24 tháng 06. năm 2026

Người nhận xét



PGS. TS. PHẠM HỒNG LIÊN

LỜI CAM KẾT

Tên đề tài: Thiết Kế Và Xây Dựng Robot Phục Vụ Tự Hành Ứng Dụng ROS2

GVHD: PGS. TS. Phạm Hồng Liên

Sinh viên 1:

- Họ tên sinh viên: Vũ Tấn Khoa
- MSSV: 17119086
- Địa chỉ: 40/19/3E đường 4 phường Tam Phú, Thủ Đức, TP Hồ Chí Minh
- Số điện thoại: 0325714520
- Email: Vutankhoa413@gmail.com

Sinh viên 2 :

- Họ tên sinh viên: Ngô Quốc Khánh
- MSSV: 22161269
- Địa chỉ: 72/2/1, đường làng Tăng Phú, Tăng Nhơn Phú A, Thủ Đức, TP Hồ Chí Minh
- Số điện thoại: 0985045485
- Email: ngoquockhanh706@gmail.com

Ngày nộp khóa luận tốt nghiệp (ĐATN): 26/6/2026

Lời cam kết: "Chúng tôi xin cam đoan khóa luận tốt nghiệp này là công trình do chính chúng tôi nghiên cứu và thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn. Chúng tôi không sao chép từ bất kỳ công trình nào đã được công bố mà không trích dẫn nguồn gốc rõ ràng. Nếu có bất kỳ vi phạm nào, chúng tôi xin hoàn toàn chịu trách nhiệm."

Tp. Hồ Chí Minh, ngày ... tháng ... năm 2026

Ký tên

LỜI CẢM ƠN

Đề tài “Thiết Kế Và Xây Dựng Robot Phục Vụ Tự Hành Ứng Dụng ROS2” là kết quả của quá trình học tập, nghiên cứu và thực hiện đồ án tốt nghiệp của nhóm tại Khoa Điện. Trong suốt thời gian triển khai đề tài, nhóm đã có cơ hội vận dụng những kiến thức đã được trang bị trên giảng đường vào thực tế, đồng thời đối mặt với nhiều khó khăn trong quá trình thiết kế phần cứng, phát triển phần mềm điều khiển và tích hợp hệ thống robot tự hành. Tuy nhiên, nhờ sự quan tâm, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè, nhóm đã từng bước hoàn thành đề tài theo đúng mục tiêu đề ra.

Trước hết, nhóm xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS. Phạm Hồng Liên, người đã trực tiếp hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đồ án. Những ý kiến đóng góp chuyên môn, sự tận tâm chỉ dẫn và những định hướng nghiên cứu của thầy đã giúp nhóm giải quyết nhiều vấn đề phát sinh cũng như hoàn thiện đề tài một cách hiệu quả hơn.

Nhóm cũng xin chân thành cảm ơn quý thầy cô thuộc “Khoa Điện” cùng toàn thể giảng viên của nhà trường đã truyền đạt những kiến thức quý báu trong suốt thời gian học tập. Những nền tảng về điện tử, hệ thống nhúng, lập trình, điều khiển và tự động hóa chính là cơ sở quan trọng giúp nhóm có đủ kiến thức và kỹ năng để thực hiện đề tài này.

Bên cạnh đó, nhóm xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên và tạo điều kiện thuận lợi cả về tinh thần lẫn vật chất trong suốt quá trình học tập và hoàn thành đồ án tốt nghiệp.

Mặc dù đã nỗ lực hết sức trong quá trình nghiên cứu và thực hiện, nhưng do hạn chế về thời gian, kinh nghiệm thực tế cũng như phạm vi của đề tài, báo cáo

chắc chắn vẫn còn những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để tiếp tục hoàn thiện và nâng cao kiến thức chuyên môn của mình.

Chúng em xin chân thành cảm ơn.

Sinh viên thực hiện

TÓM TẮT ĐỀ TÀI

Trong bối cảnh công nghệ đang ngày càng phát triển và quá trình chuyển đổi số đang diễn ra mạnh mẽ, xu hướng ứng dụng tự động hóa và robot thông minh vào lĩnh vực dịch vụ ngày càng được quan tâm nhằm nâng cao chất lượng phục vụ, tối ưu hóa nguồn nhân lực và giảm chi phí vận hành. Đặc biệt trong môi trường nhà hàng, việc sử dụng robot hỗ trợ vận chuyển thức ăn và đồ uống đến khách hàng không chỉ góp phần tăng hiệu quả hoạt động mà còn mang lại trải nghiệm mới mẻ, hiện đại cho người sử dụng.

Xuất phát từ nhu cầu đó, đề tài “THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH ỨNG DỤNG ROS2” được thực hiện với mục tiêu nghiên cứu và phát triển một mô hình robot có khả năng tự động di chuyển trong khu vực nhà hàng để thực hiện nhiệm vụ vận chuyển món ăn và đồ uống đến các bàn phục vụ. Bên cạnh việc xây dựng hệ thống robot tự hành, đề tài còn phát triển nền tảng website nhằm hỗ trợ quản lý và theo dõi robot trong quá trình phục vụ.

Kết quả thực hiện cho thấy hệ thống được xây dựng thành công với khả năng vận hành ổn định trong môi trường thử nghiệm. Robot có thể di chuyển tự động đến các vị trí được chỉ định, nhận biết và tránh né các vật cản trên đường đi, đồng thời thực hiện nhiệm vụ vận chuyển một cách an toàn và chính xác. Kết quả nghiên cứu đã góp phần tiếp tục phát triển cho các mô hình nhà hàng thông minh, thúc đẩy quá trình chuyển đổi số và ứng dụng công nghệ tự động hóa vào thực tiễn.

MỤC LỤC

LỜI CAM KẾT	i
LỜI CẢM ƠN	ii
TÓM TẮT ĐỀ TÀI	iv
MỤC LỤC.....	v
DANH MỤC CÁC TỪ VIẾT TẮT	ix
DANH MỤC HÌNH	xii
DANH MỤC BẢNG.....	xiv
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	1
1.1 Giới thiệu.....	1
1.2 Mục tiêu đề tài.....	2
1.3 Giới hạn đề tài	3
1.4 Phương pháp nghiên cứu.....	4
1.5 Đối tượng và phạm vi nghiên cứu.....	4
1.6 Bố cục quyền báo cáo	5
KẾT LUẬN CHƯƠNG 1.....	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG CÔNG NGHỆ	6
2.1 Tổng quan về robot di động tự hành	6
2.2 Robot phục vụ	7
2.3 Hệ điều hành robot ROS2	9
2.3.1 Node	10
2.3.2 Topic và Message.....	11
2.3.3 TF Tree.....	11
2.3.4 Service và Action	11
2.4 Định vị và xây dựng bản đồ đồng thời — SLAM.....	11

2.4.2 SLAM Toolbox	12
2.5 Hệ thống điều hướng Navigation2	12
2.5.1 Map Server	13
2.5.2 Planner Server	13
2.5.3 Controller Server	13
2.5.4 Behavior Tree Navigator	14
2.5.5 Costmap	14
2.6 Định vị bằng AMCL	14
2.7 Robot vi sai và động học robot	16
2.7.1 Odometry	17
2.7.2 Động học thuận (Forward Kinematics)	18
2.7.2 Động học nghịch (Reverse Kinematics)	19
2.8 ROS2 Cơ chế truyền thông giữa hệ thống quản lý và robot	19
2.8.1 Giao thức truyền thông DDS trong ROS2	19
2.8.2 Flask và SocketIO	20
2.9 Giao thức kết nối	21
2.9.1 Giao tiếp ROS	21
2.9.2 Giao tiếp UART (Serial over USB)	22
2.10 Các phần mềm hỗ trợ	23
2.10.1 Visual Studio Code	23
2.10.2 SolidWorks	24
2.10.3 Công cụ trực quan hóa RViz2	25
KẾT LUẬN CHƯƠNG 2	26
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	28
3.1 Yêu cầu hệ thống	28

3.2 Thiết kế tổng thể hệ thống.....	29
3.3 Thiết kế phần cứng robot	31
3.3.1 Thi công lắp ráp mô hình Robot	31
3.3.2 Động cơ Dc encoder JGB37-520 và Driver BTS7960	35
3.3.2.1 Động cơ Dc encoder JGB37-520	35
3.3.2.2 Driver BTS7960	39
3.3.3 Arduino Mega 2560	40
3.3.4 Raspberry Pi 4 Mode B	41
3.3.5 RPLidar A1	42
3.3.6 Khôi nguồn.....	43
3.4 Thiết kế phần mềm robot	45
3.4.1 Kiến trúc ROS2	46
3.4.2 Xây dựng bản đồ bằng SLAM	46
3.4.3 Định vị bằng AMCL	49
3.4.4 Điều hướng bằng Navigation	49
3.5. Sơ đồ và lưu đồ hoạt động của hệ thống.....	52
3.5.1 Sơ đồ cấu trúc của robot.....	52
3.5. 2 Lưu đồ giải thuật tổng quát hệ thống chung	54
3.5.3 Sơ đồ xây dựng mô hình và mô phỏng robot.....	58
3.5.4. Lưu đồ giải thuật xử lý encoder trên arduino mega2560	59
3.6 Thiết kế website	60
3.6.1 Giao diện và chức năng của wesite quản lý	60
3.6.2 Kết nối website và robot	60
KẾT THÚC CHƯƠNG 3	61
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	63

4.1 Môi trường thực nghiệm thực tế và Rviz2	63
4.2 Thực nghiệm vị trí Robot trong thực tế và trong Rviz2.....	65
4.3 Thực nghiệm mô hình robot phục vụ nhà hàng	67
4.3.1 Trường hợp 1: Robot đến bàn số 1 khi không có vật cản	68
4.3.2 Trường hợp 2: Robot đến bàn số 2 khi không có vật cản	70
4.3.3 Trường hợp 3: Robot quay về quầy phục vụ	71
4.3.4 Trường hợp 4: Robot di chuyển và tránh né khi có vật cản phía trước ...	73
4.4 Nhận xét và đánh giá.....	74
4.4.1 Đánh giá kết quả đạt được.....	74
4.4.2 Ưu điểm.....	76
4.4.3 Hạn chế.....	77
KẾT THÚC CHƯƠNG 4.....	78
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	79
5.1 Kết luận	79
5.2 Hướng phát triển	80
KẾT LUẬN CHUNG.....	80
TÀI LIỆU THAM KHẢO.....	82

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt / Ký hiệu	Tên đầy đủ / Thuật ngữ tiếng Anh	Giải thích tiếng Việt
AMCL	Adaptive Monte Carlo Localization	Thuật toán định vị Monte Carlo thích nghi, dùng để ước lượng vị trí robot trên bản đồ đã biết.
AMR	Autonomous Mobile Robot	Robot di động tự hành, có khả năng nhận thức môi trường và di chuyển đến mục tiêu.
BT	Behavior Tree	Cây hành vi, dùng trong Nav2 để điều phối quá trình điều hướng.
COM	Communication Port	Cổng giao tiếp nối tiếp dùng để truyền dữ liệu giữa máy tính/Raspberry Pi và vi điều khiển.
DC	Direct Current	Dòng điện một chiều.
DC-DC	Direct Current to Direct Current Converter	Bộ chuyển đổi điện áp một chiều sang mức điện áp một chiều khác.
DDS	Data Distribution Service	Chuẩn truyền thông phân tán được ROS2 sử dụng để các node trao đổi dữ liệu.
GPIO	General Purpose Input/Output	Các chân vào/ra đa dụng trên vi điều khiển hoặc máy tính nhúng.
ICC	Instantaneous Center of Curvature	Tâm cong tức thời trong mô hình động học robot vi sai.

LiDAR	Light Detection and Ranging	Cảm biến đo khoảng cách bằng tia laser, dùng để quét môi trường xung quanh robot.
LTS	Long Term Support	Phiên bản được hỗ trợ dài hạn, thường dùng để chỉ các bản ROS2/Ubuntu ổn định.
Nav2	Navigation2	Hệ thống điều hướng của ROS2, hỗ trợ lập kế hoạch đường đi và điều khiển robot đến mục tiêu.
PGM	Portable Gray Map	Định dạng ảnh bản đồ dạng thang xám, thường được dùng khi lưu bản đồ 2D trong ROS.
PWM	Pulse Width Modulation	Phương pháp điều chế độ rộng xung để điều khiển tốc độ động cơ.
QoS	Quality of Service	Chính sách chất lượng dịch vụ trong truyền thông ROS2/DDS.
ROS	Robot Operating System	Nền tảng phần mềm trung gian phục vụ phát triển ứng dụng robot.
ROS2	Robot Operating System 2	Phiên bản thế hệ thứ hai của ROS, dùng làm nền tảng chính cho hệ thống robot.
RPM	Revolutions Per Minute	Đơn vị đo tốc độ quay, tính bằng vòng/phút.
RPLIDAR A1	2D Laser Scanner RPLIDAR A1	Cảm biến LiDAR 2D quét 360 độ, dùng để thu dữ liệu môi trường.
RViz2	ROS Visualization 2	Công cụ trực quan hóa dữ liệu robot trong ROS2 như bản đồ, LaserScan, TF và đường đi.
SDF	Simulation Description Format	Định dạng mô tả mô hình và môi trường dùng trong mô phỏng robot.
SLAM	Simultaneous Localization and Mapping	Định vị và xây dựng bản đồ đồng thời trong môi trường chưa biết trước.
TCPROS	TCP-based ROS Transport	Cơ chế truyền thông dựa trên TCP trong ROS1.
TF	Transform	Hệ thống biến đổi tọa độ giữa các frame trong ROS/ROS2.
UART	Universal Asynchronous Receiver-Transmitter	Chuẩn giao tiếp nối tiếp không đồng bộ.

DANH MỤC HÌNH

Hình 1. 1 Robot phục vụ của KEENON trong nhà hàng	2
Hình 2.1 Robot tự phục vụ cao cấp Pudu BellaBot cho nhà hàng, quán cà phê, văn phòng, ngân hàng.....	8
Hình 2. 2 ROBOT phục vụ nhà hàng DINERBOT T5	8
Hình 2. 3 Động học bộ truyền động vi sai	16
Hình 2. 4 Giao diện Website quản lý điều khiển và theo dõi robot theo thời gian thực.....	21
Hình 2. 5 Giao diện Visual Studio Code.....	24
Hình 2. 6 Giao diện công cụ SolidWorks	25
Hình 2. 7 Giao diện công cụ trực quan hóa RViz2	26
Hình 3. 1 Sơ đồ khối tổng thể hệ thống.....	31
Hình 3. 2 Mô phỏng 3D Robot trên Solidworks	32
Hình 3. 3 Mô hình Robot thực tế và vị trí lắp đặt linh kiện	33
Hình 3. 4 Sơ đồ cấu trúc thân Robot.....	34
Hình 3. 5 Mô hình phân tích lực một bánh xe	36
Hình 3. 6 Động cơ DC Encoder JGB37-520.....	39
Hình 3. 7 Driver BTS7960	39
Hình 3. 8 Arduino Mega 2560	40
Hình 3. 9 Raspberry Pi 4.....	41
Hình 3. 10 Cảm biến LiDAR RPLidar A1	43
Hình 3. 11 Mạch nguồn XL4016	45
Hình 3. 12 Robot khi bắt đầu chạy quét map trên Rviz2	47
Hình 3. 13 Robot đã quét Map xong trong Rviz2	48
Hình 3. 14 Robot chạy quét Map trên thực tế	49
Hình 3. 15 Thiết lập hướng đi và vị trí ban đầu của robot.....	50
Hình 3. 16 Dùng Goal Pose để thiết lập hướng và vị trí đến	50
Hình 3. 17 Quá trình di chuyển của robot đến vị trí chỉ định trên Rviz2	51

Hình 3. 18 Robot đã di chuyển đến vị trí được chỉ định Trên Rviz2.....	51
Hình 3. 19 sơ đồ cấu trúc thân Robot.....	52
Hình 3. 20 Lưu đồ giải thuật hệ thống chung	54
Hình 3. 21 sơ đồ chế độ quét map.....	55
Hình 3. 22 Lưu đồ giải thuật chế độ quét và điều khiển robot.....	56
Hình 3. 23 sơ đồ chi tiết các bước ở chế độ điều hướng.....	57
Hình 3. 24 sơ đồ xây dựng mô hình và mô phỏng Robot	58
Hình 3. 25 lưu đồ thuật toán Arduino mega truyền và nhận dữ liệu	59
Hình 3. 26 Sơ đồ mô tả luồng dữ liệu giao tiếp giữa Web với ROS	61
Hình 4. 1 Kết quả đo chiều dài bản đồ trên Rviz2.....	64
Hình 4. 2 Kết quả đo chiều rộng trên Rviz2	64
Hình 4. 3 Diện tích môi trường thực tế.....	65
Hình 4. 4 vị trí quan sát được trên Rviz	66
Hình 4. 5 vị trí robot thực tế bên ngoài	67
Hình 4. 6 Giao diện xác nhận di chuyển để vị trí số 1 trước khi robot thực hiện điều hướng.....	68
Hình 4. 7 Giao diện Robot di chuyển thành công đến vị trí mục tiêu được lựa chọn trên bản đồ.....	68
Hình 4. 8 Robot đã di chuyển đến vị trí bàn số 1 trong thực tế	69
Hình 4. 9 Giao diện xác nhận di chuyển để vị trí số 1 trước khi robot thực hiện điều hướng.....	70
Hình 4. 10 Giao diện Robot di chuyển thành công đến vị trí mục tiêu được lựa chọn trên bản đồ.....	70
Hình 4. 11 Robot đã di chuyển đến vị trí bàn số 2 trong thực tế	71
Hình 4. 12 Giao diện Robot di chuyển thành công quây phục vụ trên bản đồ	72
Hình 4. 13 Robot đã đến quây ngoài thực tế.....	72
Hình 4. 14 Robot xử lý chuyển hướng lựa chọn đường đi đi né vật cản trên Rviz ..	73
Hình 4. 15 Robot chuyển hướng ngoài thực tế	74

DANH MỤC BẢNG

Bảng 2. 1 So sánh ROS 1 và ROS 2	9
Bảng 3. 1 Tóm tắt yêu cầu hệ thống	28
Bảng 3. 2 Thông số các loại động cơ dẫn động:	38
Bảng 3. 3 Thông số Raspberry Pi 4 Mode B	41
Bảng 3. 4 Thông số cảm biến LiDAR RPLidar A1M81	43
Bảng 3. 5 Thông số DC-DC XL4016	45
Bảng 3. 6 Các node chính trong kiến trúc phần mềm	46
Bảng 4. 1 Kết quả kiểm thử môi trường thực tế so với	65

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

Chương 1 trình bày tổng quan về đề tài nghiên cứu robot phục vụ nhà hàng tự hành ứng dụng ROS2. Nội dung chương tập trung phân tích bối cảnh phát triển của robot dịch vụ trong đời sống hiện nay, đặc biệt là trong lĩnh vực nhà hàng và khách sạn. Bên cạnh đó, chương cũng làm mục tiêu nghiên cứu của đề tài, phạm vi thực hiện và phương pháp triển khai. Những nội dung này đóng vai trò định hướng cho toàn bộ quá trình nghiên cứu, thiết kế và xây dựng hệ thống robot trong các chương tiếp theo.

1.1 Giới thiệu

Trong những năm gần đây, robot di động tự hành ngày càng được nghiên cứu và ứng dụng trong nhiều lĩnh vực như logistics, y tế, giáo dục, nhà hàng và các không gian dịch vụ công cộng. Điểm chung của các hệ thống này là robot không chỉ di chuyển theo lệnh đơn giản, mà còn cần nhận biết môi trường xung quanh, xác định vị trí của bản thân và thực hiện nhiệm vụ trong không gian có nhiều vật cản.

Đối với lĩnh vực nhà hàng, robot phục vụ là một hướng ứng dụng có tính thực tiễn cao. Robot có thể hỗ trợ nhân viên trong các công việc lặp lại như vận chuyển món ăn, đưa vật dụng đến bàn cho khách hàng. Tuy nhiên, để robot có thể hoạt động trong môi trường nhà hàng, hệ thống phải giải quyết được các bài toán cốt lõi như xây dựng bản đồ, định vị, điều hướng đến vị trí mong muốn và giao tiếp với hệ thống quản lý.

Đề tài “THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH ỨNG DỤNG ROS2” được thực hiện theo hướng xây dựng một nền tảng robot di động có khả năng hoạt động trong môi trường trong nhà. Hệ thống sử dụng Raspberry Pi 4 làm bộ xử lý trung tâm, Arduino Mega 2560 làm bộ điều khiển cấp thấp, cảm biến RPLidar A1 để thu dữ liệu môi trường và động cơ DC encoder JGB37-520 để di chuyển theo cấu hình bánh xe vi sai.

Về phần mềm, đề tài sử dụng Ubuntu 22.04, ROS2 Humble, SLAM Toolbox, Navigation2 và RViz2 để triển khai các chức năng nhận thức không gian, định vị và điều hướng. Bên cạnh đó, hệ thống có thêm phần website phục vụ mục tiêu quản lý

và tương tác với robot trong mô hình nhà hàng thông minh. Việc kết hợp robot, ROS2, SLAM, Nav2 và website tạo ra một hướng tiếp cận phù hợp cho đề án tốt nghiệp, vừa có tính kỹ thuật hệ thống nhưng, vừa có khả năng mở rộng thành ứng dụng thực tế.



Hình 1. 1 Robot phục vụ của KEENON trong nhà hàng

1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là thiết kế và xây dựng một mô hình robot phục vụ tự hành có khả năng nhận biết môi trường trong nhà, xây dựng bản đồ bằng SLAM, định vị trên bản đồ, điều hướng đến các vị trí được chỉ định và hỗ trợ tương tác thông qua giao diện website. Từ mục tiêu tổng quát này, đề tài được triển khai theo các mục tiêu cụ thể sau:

- Thiết kế và thi công mô hình robot di động dạng bánh xe vi sai, sử dụng động cơ DC encoder JGB37-520, driver BTS7960, Arduino Mega 2560, Raspberry Pi 4, RPLidar A1, pin 18650 và mạch hạ áp XL4016.
- Triển khai nền tảng ROS2 Humble trên Ubuntu 22.04 để tổ chức các node phần mềm, topic truyền thông, dữ liệu TF và các luồng xử lý cảm biến.

- Thu nhận dữ liệu quét môi trường từ RPLidar A1 và dữ liệu chuyển động từ encoder để phục vụ quá trình tính toán odometry.
- Xây dựng bản đồ môi trường trong nhà bằng SLAM Toolbox, hiển thị dữ liệu LaserScan, odometry và bản đồ trên RViz2.
- Triển khai nền tảng định vị và điều hướng bằng Navigation2, trong đó AMCL được sử dụng để xác định vị trí robot trên bản đồ đã lưu.
- Thiết kế website phục vụ mô hình quản lý nhà hàng, giao diện quản lý và mô hình trao đổi dữ liệu giữa website, server và robot.
- Đánh giá kết quả hoạt động của hệ thống thông qua các thử nghiệm xây dựng bản đồ, định vị, điều hướng và giao diện website.

1.3 Giới hạn đề tài

Do giới hạn về thời gian, kinh phí và điều kiện thử nghiệm, đề tài được triển khai ở quy mô mô hình nghiên cứu trong nhà. Môi trường thử nghiệm là không gian phẳng, có vật cản cố định hoặc thay đổi ở mức đơn giản, phù hợp với khả năng cảm biến của LiDAR 2D và robot bánh xe vi sai.

Về chức năng robot, hệ thống tập trung vào các nhiệm vụ chính gồm thu nhận dữ liệu LiDAR, đọc encoder, tính toán odometry, xây dựng bản đồ 2D, định vị trên bản đồ và điều hướng trong phạm vi môi trường thử nghiệm. Các chức năng như nhận dạng khuôn mặt, nhận dạng giọng nói, xử lý ảnh bằng camera, tối ưu nhiều robot hoặc vận hành thương mại quy mô lớn không thuộc phạm vi triển khai chính của đề án.

Về website, báo cáo trình bày giao diện và mô hình tương tác giữa quản lý và robot, server và robot theo hướng nhà hàng thông minh. Phần kết nối trực tiếp giữa website và robot được mô tả theo kiến trúc thiết kế và mức độ tích hợp thực tế của nhóm. Các chức năng thương mại chuyên sâu như giao tiếp khách hàng, thanh toán trực tuyến, quản lý kho, quản lý nhân sự hoặc đồng bộ với hệ thống nhà hàng lớn được xem là hướng phát triển.

1.4 Phương pháp nghiên cứu

Đề tài được thực hiện bằng phương pháp kết hợp giữa nghiên cứu lý thuyết, thiết kế hệ thống, triển khai thực nghiệm và đánh giá kết quả. Ở giai đoạn đầu, nhóm nghiên cứu tài liệu về robot di động tự hành, ROS2, SLAM, Navigation2, AMCL, LiDAR, encoder và các mô hình website quản lý trong nhà hàng.

Trên cơ sở lý thuyết, nhóm lựa chọn kiến trúc phần cứng và phần mềm phù hợp với điều kiện thực hiện. Phần cứng được thiết kế theo mô hình phân tầng: Raspberry Pi 4 xử lý dữ liệu cấp cao, Arduino Mega 2560 điều khiển động cơ và đọc encoder. Phần mềm được tổ chức theo kiến trúc ROS2, mỗi chức năng chính được tách thành node riêng để dễ kiểm thử và mở rộng.

Giai đoạn thực nghiệm tập trung kiểm tra từng khối trước khi tích hợp toàn hệ thống. Robot được kiểm tra điều khiển động cơ, đọc encoder, truyền dữ liệu Serial, hiển thị LaserScan, xuất bản odometry, chạy SLAM Toolbox, nạp bản đồ, định vị bằng AMCL và thử nghiệm quy trình điều hướng với Nav2. Website được kiểm tra theo giao diện quản lý và theo dõi robot trong mô hình nhà hàng.

1.5 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống robot di động tự hành phục vụ trong nhà, tích hợp phần cứng nhúng, cảm biến LiDAR, động cơ encoder, nền tảng ROS2 và giao diện website. Phạm vi nghiên cứu tập trung vào ba lớp kỹ thuật chính: lớp phần cứng robot, lớp phần mềm điều khiển - nhận thức robot và lớp giao diện quản lý tương tác.

Ở lớp phần cứng, đề tài nghiên cứu cấu trúc robot bánh xe vi sai, nguyên lý điều khiển động cơ DC qua driver BTS7960, phương pháp đọc encoder bằng Arduino Mega 2560, giao tiếp giữa Arduino và Raspberry Pi 4 và cũng như về yêu cầu cấp nguồn cho hệ thống di động sử dụng pin 18650 và mạch hạ áp XL4016.

Ở lớp phần mềm robot, đề tài nghiên cứu ROS2 Humble, các cơ chế truyền thông topic, message, TF, odometry, SLAM Toolbox, Navigation2 và AMCL. Các nội dung này được triển khai để robot có thể thu nhận dữ liệu môi trường, xây dựng bản đồ, xác định vị trí và điều hướng trong bản đồ.

Ở lớp giao diện, đề tài nghiên cứu cách tổ chức website quản lý nhà hàng, giao diện quản lý và mô hình kết nối giữa website, server, ROS2 và robot. Đây là cơ sở để mở rộng robot từ mô hình kỹ thuật sang mô hình ứng dụng phục vụ.

1.6 Bố cục quyền báo cáo

Báo cáo được tổ chức thành năm chương chính, trình bày theo trình tự từ tổng quan, cơ sở lý thuyết, thiết kế hệ thống đến kết quả thực nghiệm và hướng phát triển.

- Chương 1 trình bày bối cảnh, mục tiêu, giới hạn, phương pháp và phạm vi nghiên cứu của đề tài.
- Chương 2 trình bày cơ sở lý thuyết về robot di động tự hành, robot phục vụ nhà hàng, ROS2, SLAM, Navigation2, AMCL, phần cứng sử dụng, website quản lý và RViz2.
- Chương 3 trình bày quá trình thiết kế và xây dựng hệ thống, bao gồm yêu cầu hệ thống, thiết kế tổng thể, phần cứng robot, phần mềm ROS2 và website.
- Chương 4 trình bày môi trường thực nghiệm, kết quả xây dựng bản đồ, định vị, điều hướng, website và đánh giá hệ thống.
- Chương 5 tổng kết những kết quả đã đạt được, nêu hạn chế và đề xuất hướng phát triển tiếp theo.

KẾT LUẬN CHƯƠNG 1

Qua chương này, nhóm đã trình bày được tổng quan về đề tài, nêu bật nhu cầu ứng dụng robot phục vụ trong lĩnh vực nhà hàng cũng như ý nghĩa thực tiễn của hệ thống. Đồng thời, các mục tiêu, phạm vi nghiên cứu và phương pháp thực hiện cũng đã được xác định rõ ràng. Đây là cơ sở quan trọng để tiến hành nghiên cứu các nền tảng lý thuyết và công nghệ cần thiết cho việc xây dựng hệ thống robot tự hành ở các chương tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG CÔNG NGHỆ

Chương 2 trình bày các cơ sở lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống robot phục vụ nhà hàng tự hành. Nội dung chương tập trung tìm hiểu về robot di động tự hành, hệ điều hành robot ROS2, thuật toán SLAM, phương pháp định vị AMCL, hệ thống điều hướng Nav2 và mô hình động học robot vi sai. Ngoài ra, chương cũng giới thiệu các thành phần phần cứng và phần mềm liên quan đến đề tài. Những kiến thức này là nền tảng quan trọng cho việc thiết kế, triển khai và vận hành hệ thống robot trong thực tế.

2.1 Tổng quan về robot di động tự hành

Robot di động tự hành (Autonomous Mobile Robot — AMR) là hệ thống robot có khả năng di chuyển trong môi trường thực tế mà không cần con người can thiệp trực tiếp vào từng bước điều khiển. Khác với robot công nghiệp cố định hoặc robot di chuyển theo quỹ đạo lập trình sẵn, AMR có khả năng nhận thức môi trường xung quanh thông qua cảm biến, tự xây dựng hoặc sử dụng bản đồ không gian, lập kế hoạch đường đi và thực thi di chuyển một cách linh hoạt ngay cả khi môi trường thay đổi.[1]

Về mặt kiến trúc, một hệ thống AMR hoàn chỉnh thường được tổ chức theo mô hình phân tầng gồm ba lớp chức năng. Lớp nhận thức (perception layer) chịu trách nhiệm thu thập và xử lý dữ liệu từ các cảm biến như LiDAR, camera hay encoder để xây dựng mô hình môi trường. Lớp quyết định (decision layer) thực hiện các thuật toán định vị, lập kế hoạch đường đi và ra quyết định điều hướng dựa trên mô hình môi trường đã có. Lớp thực thi (execution layer) chuyển đổi các quyết định điều hướng thành tín hiệu điều khiển cụ thể cho cơ cấu chấp hành.

Trong lĩnh vực dịch vụ, AMR ngày càng được triển khai rộng rãi nhờ khả năng vận hành liên tục và có thể tích hợp với các hệ thống quản lý số. Trong phạm vi đề tài, hệ thống robot được thiết kế và triển khai theo đúng mô hình AMR ba lớp nêu trên, với Raspberry Pi 4 đảm nhiệm lớp nhận thức và quyết định, Arduino Mega 2560 đảm nhiệm lớp thực thi.

2.2 Robot phục vụ

Robot phục vụ là một dạng AMR được thiết kế chuyên biệt để hỗ trợ hoạt động phục vụ trong môi trường ăn uống, thực hiện các nhiệm vụ như vận chuyển thức ăn và đồ uống từ quầy bếp đến bàn khách hàng, thu dọn bát đĩa sau khi khách ăn xong, hoặc hướng dẫn khách đến bàn. Điểm khác biệt so với robot công nghiệp là robot phục vụ phải hoạt động trong không gian chung với con người, đòi hỏi khả năng phát hiện và tránh vật cản động, di chuyển với vận tốc an toàn và giao diện tương tác thân thiện.

Về mô hình hoạt động, robot phục vụ thường vận hành theo chu trình: nhận nhiệm vụ từ hệ thống quản lý hoặc nhân viên, vận chuyển đến bàn đích được chỉ định, xác nhận hoàn thành và về vị trí chờ. Để thực hiện chu trình này, robot cần tích hợp đầy đủ các năng lực: định vị trong bản đồ đã biết, lập kế hoạch đường đi tối ưu, tránh vật cản trong thời gian thực và giao tiếp với hệ thống quản lý bên ngoài.

Trên thị trường hiện nay, một số robot phục vụ thương mại tiêu biểu đã được triển khai rộng rãi, điển hình là BellaBot và KettyBot của Pudu Robotics (Trung Quốc), DinerBot T5 của LG Electronics (Hàn Quốc), và Servi của Bear Robotics (Hoa Kỳ). Các hệ thống này sử dụng LiDAR kết hợp camera độ sâu để nhận thức môi trường, tích hợp màn hình cảm ứng để tương tác với khách hàng và có thể mang được tải trọng từ 10 đến 40 kg. Tuy nhiên, chi phí đầu tư và bảo trì của các sản phẩm thương mại này còn tương đối cao, tạo tiền đề cho hướng nghiên cứu phát triển hệ thống tự chế dựa trên nền tảng phần cứng mở và phần mềm mã nguồn mở như ROS2.[2]



Hình 2.1 Hình 2. 1 Robot tự phục vụ cao cấp Pudu BellaBot cho nhà hàng, quán cà phê, văn phòng, ngân hàng



Hình 2. 2 ROBOT phục vụ nhà hàng DINERBOT T5

2.3 Hệ điều hành robot ROS2

ROS2 (Robot Operating System 2) là nền tảng phần mềm trung gian mã nguồn mở thế hệ thứ hai, được thiết kế chuyên biệt cho các ứng dụng robot phân tán, yêu cầu độ tin cậy cao và hỗ trợ môi trường đa nền tảng. Khác với phiên bản tiền nhiệm ROS1, ROS2 được xây dựng trên nền tảng truyền thông DDS (Data Distribution Service), cho phép các tiến trình phần mềm tự khám phá và thiết lập kết nối ngang hàng mà không cần một máy chủ trung tâm. Điều này làm tăng đáng kể tính ổn định và khả năng mở rộng của hệ thống.[3]

Bảng 2. 1 So sánh ROS 1 và ROS 2

Đặc điểm	ROS 1	ROS 2
Thời điểm ra mắt	2007	2017
Middleware truyền thông	TCPROS/UDPROS (tùy chỉnh)	DDS (Data Distribution Service) — chuẩn công nghiệp
Máy chủ trung tâm	Bắt buộc cần ROS Master (roscore)	Không cần — peer-to-peer tự khám phá
Hỗ trợ thời gian thực (Real-time)	Không	Có (nhờ DDS và chính sách QoS)
Chính sách QoS	Không có	Có (reliable, best-effort, transient local...)
Quản lý vòng đời Node	Không	Có (Lifecycle Nodes)
Ngôn ngữ lập trình	C++, Python 2/3	C++, Python 3, Java, Rust

Đặc điểm	ROS 1	ROS 2
Hỗ trợ Python 3 đầy đủ	Chỉ từ Noetic (2020)	Tất cả các phiên bản
Khả năng đa robot	Hạn chế, cần cấu hình phức tạp	Tốt, hỗ trợ tự nhiên qua DDS domain
Bảo mật	Không có	Có (SROS2 — mã hóa, xác thực)
Hệ điều hành hỗ trợ	Chủ yếu Ubuntu Linux	Ubuntu, Windows 10/11, macOS
Phiên bản LTS mới nhất	Noetic Ninjemys (EOL 2025)	Humble Hawksbill (LTS đến 2027)
Trạng thái phát triển	Ngừng phát triển sau Noetic	Vẫn đang phát triển và còn được hỗ trợ .
Ứng dụng trong đề tài	Không sử dụng	ROS2 Humble — toàn bộ stack phần mềm

Trong đề tài, ROS2 Humble Hawksbill được lựa chọn vì đây là phiên bản LTS (Long Term Support) được hỗ trợ chính thức trên Ubuntu 22.04, có hệ sinh thái ổn định và được sử dụng phổ biến, tương thích với toàn bộ các gói phần mềm cần thiết bao gồm SLAM Toolbox, Navigation2 và driver RPLidar A1M81.

2.3.1 Node

Node là đơn vị thực thi cơ bản trong kiến trúc ROS2, mỗi Node là một tiến trình độc lập đảm nhiệm một chức năng chuyên biệt. Kiến trúc đa Node cho phép phân tách rõ ràng các chức năng, cô lập lỗi và tái sử dụng mã nguồn. Trong hệ thống của đề tài, các Node chính bao gồm driver RPLidar A1M81, Node giao tiếp Serial với Arduino, Node teleop, SLAM Toolbox, các Node của Navigation2 và Node RViz2.

2.3.2 Topic và Message

Topic là kênh truyền thông bất đồng bộ theo mô hình publish/subscribe, cho phép các Node trao đổi dữ liệu mà không cần biết danh tính của nhau. Mỗi Topic được định kiểu nghiêm ngặt bằng một loại Message cụ thể, đảm bảo tính nhất quán dữ liệu. Trong hệ thống của đề tài, các Topic quan trọng gồm /scan mang dữ liệu sensor_msgs/LaserScan từ RPLidar A1, /odom mang dữ liệu nav_msgs/Odometry từ Node encoder, /cmd_vel mang lệnh vận tốc geometry_msgs/Twist từ Navigation2 đến Arduino, và /map mang dữ liệu nav_msgs/OccupancyGrid từ SLAM Toolbox hoặc Map Server.

2.3.3 TF Tree

TF (Transform) là hệ thống quản lý mối quan hệ hình học giữa các hệ tọa độ trong ROS2. Mỗi thành phần vật lý của robot và không gian làm việc được đại diện bởi một frame tọa độ, và TF lưu trữ các phép biến đổi giữa các frame này theo thời gian. Trong hệ thống của đề tài, TF tree bao gồm bốn frame chính theo chuỗi map → odom → base_link → laser_frame. Frame map do SLAM Toolbox hoặc AMCL broadcast, biểu diễn hệ tọa độ toàn cục của bản đồ. Frame odom là hệ tọa độ odometry tích lũy từ vị trí khởi động. Frame base_link gắn với thân robot. Frame laser_frame gắn với RPLidar A1M81 và kết nối với base_link qua transform tĩnh nhằm xác định vị trí lắp đặt cảm biến.

2.3.4 Service và Action

Bên cạnh Topic, ROS2 cung cấp hai cơ chế giao tiếp bổ sung. Service là cơ chế giao tiếp đồng bộ theo mô hình yêu cầu/phản hồi, phù hợp với các tác vụ ngắn có kết quả xác định như nạp bản đồ hay lưu bản đồ. Action là cơ chế giao tiếp bất đồng bộ phù hợp với các tác vụ dài hơi như điều hướng đến đích, cho phép theo dõi tiến trình và hủy giữa chừng. Navigation2 sử dụng Action để nhận Goal Pose từ người dùng hoặc hệ thống bên ngoài và phản hồi trạng thái thực thi trong suốt quá trình di chuyển.

2.4 Định vị và xây dựng bản đồ đồng thời — SLAM

2.4.1 Khái niệm SLAM

SLAM (Simultaneous Localization and Mapping) là công nghệ cho phép robot di chuyển trong môi trường chưa biết trước, đồng thời xây dựng bản đồ không gian và ước lượng vị trí của chính mình trong bản đồ đó theo thời gian thực. Hai tiến trình Mapping và Localization diễn ra song song và tương hỗ trợ bản đồ được cập nhật dựa trên vị trí ước lượng hiện tại, trong khi vị trí được hiệu chỉnh dựa trên đặc trưng hình học của bản đồ đang xây dựng.[4]

Kết quả đầu ra của SLAM là bản đồ Occupancy Grid Map — một ma trận hai chiều trong đó mỗi ô lưới mang giá trị biểu thị xác suất bị chiếm chỗ của vùng không gian tương ứng. Các ô có giá trị cao (màu đen trên RViz2) biểu thị vật cản cố định như tường và đồ vật. Các ô có giá trị thấp (màu trắng) biểu thị không gian tự do. Các ô chưa được cảm biến quét tới (màu xám) mang giá trị không xác định. Cấu trúc dữ liệu này không chỉ là kết quả của quá trình SLAM mà còn là đầu vào bắt buộc cho hệ thống điều hướng Navigation2 ở giai đoạn sau.

2.4.2 SLAM Toolbox

SLAM Toolbox là gói phần mềm SLAM mã nguồn mở được tích hợp chính thức trong hệ sinh thái ROS2, sử dụng thuật toán Graph-based SLAM với khả năng xử lý theo thời gian thực trên phần cứng nhúng tầm trung. Điểm mạnh của SLAM Toolbox so với các gói SLAM khác là hỗ trợ chế độ mapping liên tục (lifelong mapping), lưu và nạp lại trạng thái bản đồ, và tích hợp sẵn khả năng phát hiện đóng vòng lặp (loop closure) để giảm sai số tích lũy trong quá trình quét không gian lớn.

Trong đề tài, SLAM Toolbox hoạt động ở chế độ online asynchronous mapping: nhận dữ liệu từ topic /scan và thông tin biến đổi tọa độ từ TF tree, cập nhật bản đồ và broadcast transform map → odom liên tục trong khi robot di chuyển dưới sự điều khiển bàn phím. Sau khi quá trình quét hoàn tất, bản đồ được lưu ra tệp .pgm và .yaml để sử dụng cho giai đoạn điều hướng Navigation2.[5]

2.5 Hệ thống điều hướng Navigation2

Navigation2 (Nav2) là stack điều hướng chính thức của ROS2, cung cấp đầy đủ các thành phần cần thiết để một robot di động có thể tự lập kế hoạch và thực hiện di chuyển từ vị trí hiện tại đến đích được chỉ định, đồng thời tránh các vật cản tĩnh và

động trong quá trình di chuyển. Nav2 được thiết kế theo kiến trúc plugin, cho phép thay thế linh hoạt từng thành phần mà không ảnh hưởng đến toàn bộ hệ thống.[6]

Luồng xử lý cơ bản của Nav2 đi từ Goal Pose do người dùng hoặc hệ thống bên ngoài cung cấp, qua BT Navigator điều phối toàn bộ quá trình, đến Planner Server tính toán đường đi toàn cục, tiếp đến Controller Server tạo lệnh vận tốc tức thời, và cuối cùng là topic `/cmd_vel` truyền lệnh điều khiển xuống phần cứng robot.[7]

2.5.1 Map Server

Map Server là thành phần chịu trách nhiệm nạp bản đồ đã lưu từ tệp `.pgm` và `.yaml` vào hệ thống ROS2 và publish lên topic `/map` dưới dạng `nav_msgs/OccupancyGrid`. Bản đồ này được Nav2 sử dụng làm nền tảng để xây dựng costmap toàn cục và cung cấp cho AMCL làm mốc tham chiếu định vị. Trong đề tài, Map Server được khởi chạy cùng với Nav2 mỗi khi hệ thống chuyển từ chế độ mapping sang chế độ navigation và nạp bản đồ đã được lưu từ giai đoạn SLAM Toolbox.

2.5.2 Planner Server

Planner Server chịu trách nhiệm lập kế hoạch đường đi toàn cục (global path planning) từ vị trí hiện tại của robot đến Goal Pose trên bản đồ. Planner nhận đầu vào là vị trí robot hiện tại từ AMCL, Goal Pose từ BT Navigator và bản đồ toàn cục từ Map Server, sau đó tính toán một chuỗi điểm waypoint tạo thành đường đi ngắn nhất tránh các vật cản tĩnh trên bản đồ. Kết quả đường đi toàn cục này được chuyển sang Controller Server để thực thi.

2.5.3 Controller Server

Controller Server chịu trách nhiệm điều hướng cục bộ (local navigation), thực thi đường đi đã lập kế hoạch trong điều kiện thực tế. Controller nhận đường đi toàn cục từ Planner Server và dữ liệu cảm biến thời gian thực, tính toán vector vận tốc tuyến tính và góc tối ưu tại mỗi thời điểm để robot bám theo đường đi trong khi tránh các vật cản xuất hiện động trong môi trường. Kết quả đầu ra là lệnh vận tốc `geometry_msgs/Twist` publish lên topic `/cmd_vel`, được Node giao tiếp Serial nhận và truyền xuống Arduino Mega 2560 để điều khiển động cơ.

2.5.4 Behavior Tree Navigator

BT Navigator (Behavior Tree Navigator) là thành phần điều phối trung tâm của Nav2, sử dụng cấu trúc cây hành vi (Behavior Tree) để quản lý luồng thực thi điều hướng. Thay vì một chuỗi lệnh cứng nhắc, BT Navigator cho phép định nghĩa linh hoạt các chiến lược ứng xử khi gặp các tình huống khác nhau như không tìm được đường đi, robot bị kẹt hay Goal Pose không hợp lệ. Trong đề tài, BT Navigator sử dụng cây hành vi mặc định của Nav2, điều phối toàn bộ vòng lặp: nhận Goal Pose, gọi Planner, gọi Controller, theo dõi tiến trình và xác nhận hoàn thành khi robot đến đích.

2.5.5 Costmap

Costmap là cấu trúc dữ liệu bổ sung mà Nav2 sử dụng song song với bản đồ Occupancy Grid để biểu diễn không gian an toàn cho robot di chuyển. Nav2 duy trì hai lớp costmap: costmap toàn cục (global costmap) dựa trên bản đồ tĩnh từ Map Server, phục vụ lập kế hoạch đường đi dài hạn; và costmap cục bộ (local costmap) cập nhật theo thời gian thực từ dữ liệu LiDAR, phục vụ tránh vật cản trong phạm vi gần. Mỗi ô trong costmap mang giá trị chi phí (cost) phản ánh mức độ nguy hiểm khi robot đi qua vùng đó, từ hoàn toàn tự do đến bị chặn hoàn toàn, với vùng đệm lạm phát (inflation layer) xung quanh các vật cản để đảm bảo khoảng cách an toàn cho thân robot.

2.6 Định vị bằng AMCL

AMCL (Adaptive Monte Carlo Localization) là thuật toán định vị xác suất cho phép robot xác định vị trí và hướng của mình trên một bản đồ đã biết trước bằng cách đối chiếu dữ liệu cảm biến thực tế với bản đồ tham chiếu. AMCL là thành phần định vị mặc định trong Nav2 và được sử dụng ở giai đoạn điều hướng sau khi bản đồ đã được xây dựng bởi SLAM Toolbox.

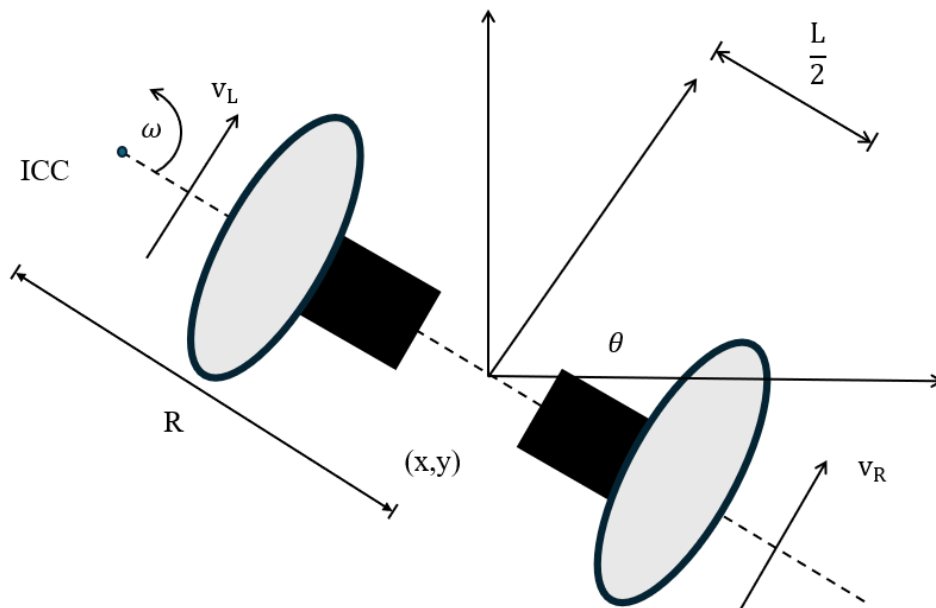
Nguyên lý hoạt động của AMCL dựa trên thuật toán Particle Filter (bộ lọc hạt): hệ thống duy trì một tập hợp các hạt (particle), mỗi hạt đại diện cho một giả thuyết về vị trí và hướng có thể có của robot. Ban đầu, các hạt được phân bố trên toàn bộ bản đồ hoặc tập trung tại vùng ước lượng ban đầu do người dùng cung cấp. Tại mỗi bước

thời gian, thuật toán thực hiện ba giai đoạn: cập nhật dự đoán dựa trên dữ liệu odometry, cập nhật quan sát bằng cách tính trọng số cho từng hạt dựa trên mức độ khớp giữa dữ liệu LaserScan thực tế và vị trí tương ứng trên bản đồ, và bước tái lấy mẫu (resampling) loại bỏ các hạt có trọng số thấp và nhân bản các hạt có trọng số cao. Qua nhiều vòng lặp, các hạt hội tụ về vùng vị trí thực tế của robot, cho phép AMCL ước lượng chính xác vị trí robot trên bản đồ.[8]

Trong hệ thống của đề tài, AMCL nhận đầu vào từ ba nguồn song song: dữ liệu LaserScan từ topic /scan, thông tin odometry qua TF frame odom $\rightarrow$ base_link, và bản đồ tĩnh từ topic /map. Đầu ra của AMCL là ước lượng vị trí robot dưới dạng geometry_msgs/PoseWithCovarianceStamped publish lên topic /amcl_pose, đồng thời broadcast transform map $\rightarrow$ odom lên TF tree để toàn bộ hệ thống Nav2 có thể sử dụng vị trí robot trong hệ tọa độ bản đồ.

Điểm quan trọng cần lưu ý là AMCL yêu cầu người dùng cung cấp ước lượng vị trí ban đầu (initial pose) thông qua RViz2 hoặc topic /initialpose khi khởi động, vì thuật toán cần điểm xuất phát để hội tụ nhanh hơn. Sau khi hội tụ, AMCL duy trì định vị ổn định ngay cả khi robot di chuyển liên tục trong không gian.[8]

2.7 Robot vi sai và động học robot



Hình 2. 3 Động học bộ truyền động vi sai

Để thực hiện chuyển động lăn, robot quay quanh một điểm trên mặt phẳng được gọi là Tâm cong tức thời (ICC). Điểm này nằm trên trục bánh xe và xác định độ cong của quỹ đạo tại một thời điểm nhất định. Để cả hai bánh xe cùng quay quanh một tâm ICC, tốc độ quay của chúng phải tương thích với nhau [9]. Các phương trình sau đây mô tả mối quan hệ này:

$$\omega (R + L/2) = V_R \text{ (đánh số công thức)}$$

$$\omega (R - L/2) = V_L \text{ (đánh số công thức)}$$

Trong đó:

- L : là khoảng cách giữa tâm hai bánh xe (m)
- V_R, V_L : là vận tốc tịnh tiến của bánh phải và bánh trái (m/s)
- R : là khoảng cách từ tâm quay tức thời (ICC) đến trung điểm của trục bánh xe (m)
- ω : là vận tốc góc của robot quanh tâm quay tức thời (ICC) (rad/s)

2.7.1 Odometry

Odometry là phương pháp ước lượng vị trí và hướng của robot dựa trên dữ liệu thu được từ các cảm biến chuyển động, phổ biến nhất là encoder gắn trên bánh xe. Trong robot di động tự hành, Odometry đóng vai trò quan trọng trong việc xác định quãng đường đã di chuyển và trạng thái hiện tại của robot theo thời gian thực. Thông tin odometry được sử dụng làm dữ liệu đầu vào cho các thuật toán định vị và điều hướng như AMCL và Navigation2 nhằm hỗ trợ robot xác định vị trí và di chuyển đến mục tiêu mong muốn.

Trong hệ thống được xây dựng, mỗi động cơ được trang bị encoder để đo số xung quay của bánh xe. Từ số lượng xung encoder thu được trong một khoảng thời gian xác định, quãng đường di chuyển của từng bánh xe được tính theo công thức

$$d_L = N_L \frac{2\pi R}{N_{rev}}$$

$$d_R = N_R \frac{2\pi R}{N_{rev}}$$

Trong đó:

- d_L, d_R là quãng đường di chuyển của bánh trái và bánh phải (m).
- N_L, N_R là số xung encoder đo được của bánh trái và bánh phải.
- R là bán kính bánh xe (m).
- N_{rev} là số xung encoder trong một vòng quay của bánh xe.

Từ quãng đường di chuyển của hai bánh, vận tốc tuyến tính và vận tốc góc của robot được xác định dựa trên mô hình động học thuần của robot vi sai. Và để cập nhật tọa độ và góc định hướng của robot theo thời gian, đề tài sử dụng phương pháp Midpoint Integration. Phương pháp này tính toán vị trí mới của robot dựa trên góc định hướng tại điểm giữa của khoảng thời gian lấy mẫu, giúp giảm sai số tích lũy khi

robot vừa tiến vừa quay. Các phương trình cập nhật pose của robot được xác định như sau:

$$\begin{aligned}x_{k+1} &= x_k + \Delta s \cos(\theta + \frac{\Delta\theta}{2}) \\y_{k+1} &= y_k + \Delta s \sin(\theta + \frac{\Delta\theta}{2}) \\\theta_{k+1} &= \theta_k + \Delta\theta\end{aligned}$$

Trong đó:

- (x_k, y_k, θ_k) là vị trí và hướng của robot tại thời điểm hiện tại.
- $(x_k + 1, y_k + 1, \theta_k + 1)$ là vị trí và hướng của robot tại thời điểm tiếp theo.
- Δs là quãng đường robot di chuyển trong chu kỳ lấy mẫu.
- $\Delta\theta$ là góc quay của robot trong chu kỳ lấy mẫu.

2.7.2 Động học thuận (Forward Kinematics)

Động học thuận (Forward Kinematics) cho phép tính toán được vận tốc và tốc độ góc khi biết được vận tốc bánh trái, vận tốc bánh phải và khoảng cách hai bánh

$$\begin{aligned}v &= \frac{v_R + v_L}{2} \\\omega &= \frac{v_R - v_L}{L}\end{aligned}$$

Trong đó:

- v_L : vận tốc bánh trái (m/s).

- v_R : vận tốc bánh phải (m/s).
- L : khoảng cách giữa hai bánh xe (m).
- v : vận tốc tuyến tính của robot (m/s).
- ω : vận tốc góc của robot (rad/s).

2.7.2 Động học nghịch (Forward Kinematics)

Động học nghịch (Forward Kinematics) giúp chúng ta tính được vận tốc góc trái và vận tốc góc phải khi biết được vận tốc, khoảng cách tâm hai bánh và vận tốc góc từ đó để có thể xác định các góc quay, có công thức:

$$\omega_R = \frac{v + \frac{L}{2}\omega}{R}$$

$$\omega_L = \frac{v - \frac{L}{2}\omega}{R}$$

2.8 ROS2 Cơ chế truyền thông giữa hệ thống quản lý và robot

2.8.1 Giao thức truyền thông DDS trong ROS2

DDS (Data Distribution Service) là middleware được ROS2 sử dụng để trao đổi dữ liệu giữa các node. DDS cho phép các node hoạt động trên nhiều thiết bị khác nhau trong cùng mạng nội bộ mà không cần thiết lập kết nối thủ công, Raspberry Pi và máy tính chạy website cùng tham gia vào mạng ROS2 thông qua DDS và các topic như /odom, /scan, /cmd_vel và các Action của Nav2 được truyền qua mạng nhờ DDS.

2.8.2 Flask và SocketIO

Flask được sử dụng để xây dựng giao diện website quản lý. SocketIO hỗ trợ trao đổi dữ liệu hai chiều theo thời gian thực giữa trình duyệt và máy chủ web. Dữ liệu từ ROS2 được Flask tiếp nhận và gửi đến giao diện web thông qua SocketIO và các thao tác trên website được truyền ngược về ROS2 để thực hiện điều khiển robot.

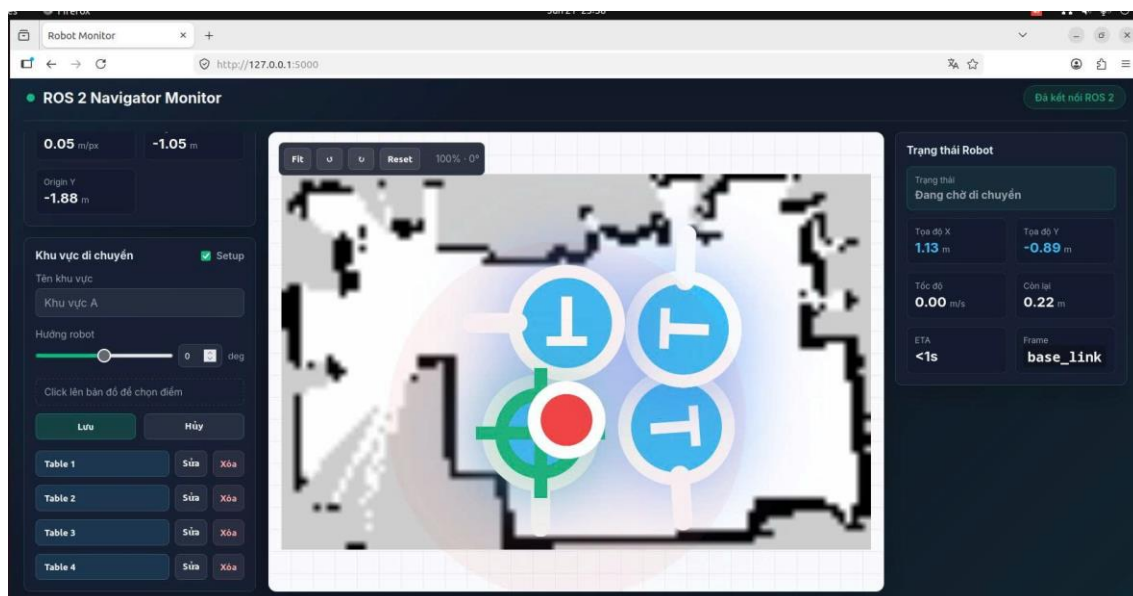
Bên cạnh khả năng tự hành và điều hướng, việc xây dựng một giao diện quản lý trực quan giúp nâng cao hiệu quả khai thác và vận hành robot trong môi trường nhà hàng. Trong đề tài này, hệ thống website được phát triển nhằm hỗ trợ người dùng theo dõi trạng thái hoạt động của robot, quản lý các vị trí phục vụ và thực hiện tương tác điều khiển từ xa thông qua mạng nội bộ.

Website đóng vai trò là cầu nối giữa người quản lý và hệ thống robot. Thông qua giao diện trực quan trên trình duyệt web, người dùng có thể quan sát bản đồ môi trường đã được xây dựng, theo dõi vị trí hiện tại của robot, trạng thái hoạt động, tốc độ di chuyển, khoảng cách còn lại đến mục tiêu và các thông tin điều hướng theo thời gian thực. Dữ liệu được cập nhật liên tục từ hệ thống ROS2, giúp đảm bảo tính đồng bộ giữa trạng thái thực tế của robot và giao diện giám sát.

Ngoài chức năng giám sát, website còn cho phép thiết lập và quản lý các điểm phục vụ (bàn ăn) trên bản đồ. Người dùng có thể lựa chọn vị trí trực tiếp trên bản đồ, lưu thông tin điểm đến và gửi lệnh điều hướng cho robot. Khi nhận được yêu cầu, robot sẽ sử dụng hệ thống Navigation2 để tự động lập kế hoạch đường đi và di chuyển đến vị trí được chỉ định. Nhờ đó, quá trình vận hành trở nên thuận tiện hơn, giảm thao tác thủ công và nâng cao khả năng ứng dụng của robot trong môi trường phục vụ nhà hàng.

Hệ thống website không chỉ hỗ trợ quản lý tập trung mà còn tạo nền tảng cho việc mở rộng các chức năng trong tương lai như đặt món trực tuyến, gọi robot phục vụ từ bàn ăn, theo dõi nhiều robot đồng thời hoặc thống kê dữ liệu vận hành. Đây là một

thành phần quan trọng góp phần hoàn thiện mô hình robot phục vụ tự hành, đáp ứng xu hướng chuyển đổi số và tự động hóa trong lĩnh vực dịch vụ hiện nay.



Hình 2. 4 Giao diện Website quản lý điều khiển và theo dõi robot theo thời gian thực

2.9 Giao thức kết nối

Hệ thống robot phục vụ tự hành yêu cầu sự phối hợp liên tục giữa nhiều thành phần phần cứng và phần mềm chạy trên các nền tảng khác nhau. Để đảm bảo dữ liệu được truyền đúng, đủ và kịp thời giữa các thành phần, hệ thống sử dụng hai giao thức kết nối chính: giao tiếp nội bộ giữa các node thông qua cơ chế topic của ROS2, và giao tiếp phần cứng giữa Raspberry Pi 4 và Arduino Mega 2560 thông qua chuẩn Serial over USB. Hai giao thức này hoạt động ở hai lớp khác nhau nhưng phối hợp chặt chẽ để tạo thành luồng dữ liệu thông suốt từ cảm biến đến cơ cấu chấp hành.

2.9.1 Giao tiếp ROS

Trong kiến trúc phần mềm của hệ thống, các node ROS2 trao đổi dữ liệu với nhau thông qua cơ chế topic theo mô hình publish/subscribe. Đây là giao thức giao tiếp nội bộ chính của toàn bộ lớp phần mềm robot, hoạt động trên cùng một máy tính nhúng Raspberry Pi 4 hoặc có thể mở rộng sang nhiều thiết bị trong cùng mạng.

Mỗi topic trong hệ thống mang một loại message được định kiểu cụ thể, đảm bảo dữ liệu truyền đi luôn có cấu trúc nhất quán. Node driver RPLidar A1 publish dữ liệu quét laser lên topic /scan với kiểu sensor_msgs/LaserScan. Node odometry tính toán và publish vị trí robot lên topic /odom với kiểu nav_msgs/Odometry, đồng thời broadcast transform odom → base_link lên hệ thống TF. Navigation2 subscribe các topic này cùng với dữ liệu bản đồ từ /map, xử lý và publish lệnh vận tốc kết quả lên topic /cmd_vel với kiểu geometry_msgs/Twist.

Cơ chế publish/subscribe của ROS2 hoạt động phi đồng bộ và phi tập trung nhờ nền tảng DDS bên dưới. Các node không cần biết danh tính của nhau, chỉ cần đăng ký đúng tên topic và đúng kiểu message là có thể trao đổi dữ liệu. Điều này giúp hệ thống dễ mở rộng: bổ sung một node mới chỉ cần publish hoặc subscribe đúng topic mà không cần thay đổi các node đang chạy. Trong hệ thống của đề tài, toàn bộ luồng dữ liệu từ LiDAR qua SLAM, AMCL, Nav2 đến lệnh điều khiển đều được truyền qua cơ chế topic ROS2 này.

2.9.2 Giao tiếp UART (Serial over USB)

Giao tiếp giữa Raspberry Pi 4 và Arduino Mega 2560 được thực hiện thông qua cổng USB vật lý, tuy nhiên bản chất giao thức bên dưới là UART (Universal Asynchronous Receiver-Transmitter) được đóng gói qua lớp USB nhờ chip chuyển đổi tích hợp trên bo mạch Arduino (thường là CH340 hoặc ATmega16U2). Phía hệ điều hành Linux trên Raspberry Pi 4, kết nối này xuất hiện dưới dạng một cổng Serial ảo, thường là /dev/ttyUSB0 hoặc /dev/ttyACM0, và được truy cập như một giao tiếp UART thông thường ở tốc độ baud 115200.

UART là chuẩn truyền thông nối tiếp không đồng bộ, trong đó dữ liệu được truyền từng bit theo thứ tự trên một dây tín hiệu. Mỗi khung dữ liệu UART bao gồm bit bắt đầu (start bit), các bit dữ liệu, bit kiểm tra chẵn lẻ tùy chọn và một hoặc hai bit dừng (stop bit). Tốc độ baud xác định số bit được truyền mỗi giây; ở mức 115200 baud, mỗi byte dữ liệu được truyền trong khoảng 87 microsecond, đủ nhanh cho việc cập nhật lệnh điều khiển và dữ liệu encoder ở tần số hàng chục Hz.

Trong hệ thống của đề tài, hai chiều dữ liệu được định nghĩa rõ ràng trên kênh Serial này. Theo chiều từ Raspberry Pi 4 xuống Arduino, node cmdvel_to_pwm_serial đóng gói lệnh vận tốc nhận từ topic /cmd_vel thành frame

dạng `M,<pwmL>,<pwmR>` và gửi xuống Arduino. Arduino giải mã frame này, tách giá trị PWM của bánh trái và bánh phải, sau đó xuất tín hiệu PWM tương ứng cho hai driver BTS7960. Theo chiều từ Arduino lên Raspberry Pi 4, Arduino định kỳ gửi dữ liệu encoder thô theo frame dạng `E,<countL>,<countR>`. Node odometry trên Raspberry Pi 4 đọc frame này, tính toán quãng đường và hướng di chuyển của robot theo mô hình động học bánh xe vi sai, rồi publish kết quả lên topic `/odom`.

Việc sử dụng định dạng frame văn bản có ký tự phân cách giúp dễ debug và kiểm tra dữ liệu trực tiếp trên terminal mà không cần công cụ phân tích giao thức chuyên dụng. Cơ chế dừng an toàn cũng được tích hợp trong node `cmdvel_to_pwm_serial`: nếu không nhận được lệnh `/cmd_vel` mới trong một khoảng thời gian nhất định, node tự động gửi lệnh dừng xuống Arduino để đảm bảo robot không tiếp tục di chuyển khi mất tín hiệu điều khiển.

2.10 Các phần mềm hỗ trợ

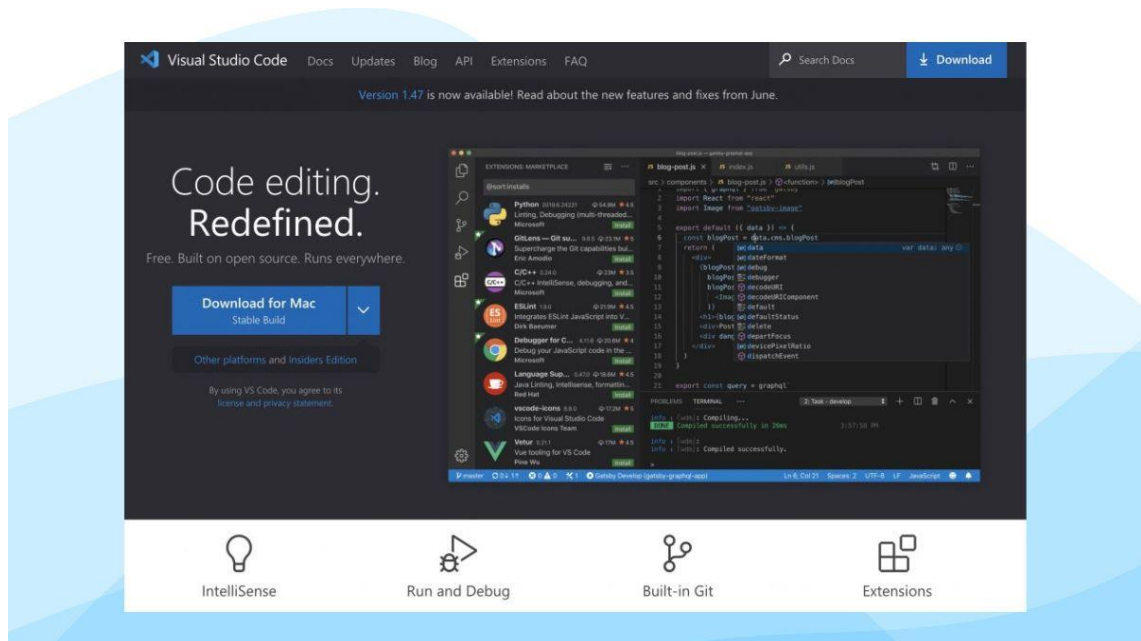
Trong quá trình thiết kế, lập trình và vận hành hệ thống robot phục vụ tự hành, đề tài sử dụng một số phần mềm công cụ hỗ trợ đóng vai trò quan trọng trong từng giai đoạn phát triển. Các phần mềm này bao gồm Visual Studio Code phục vụ lập trình và viết cấu hình, SolidWorks phục vụ thiết kế mô hình cơ khí 3D và RViz2 phục vụ trực quan hóa dữ liệu robot trong thời gian thực.

2.10.1 Visual Studio Code

Visual Studio Code (VS Code) là trình soạn thảo mã nguồn mở do Microsoft phát triển, được phát hành lần đầu vào năm 2015 và hiện là một trong những công cụ lập trình phổ biến nhất trên thế giới nhờ hiệu năng nhẹ, khả năng mở rộng linh hoạt và hỗ trợ đa nền tảng trên Windows, macOS và Linux.

Trong đề tài, VS Code được sử dụng trên máy tính phát triển chạy Ubuntu 22.04 để soạn thảo toàn bộ mã nguồn Python của các node ROS2, bao gồm node đọc encoder và tính toán odometry (`odom.py`), node chuyển đổi lệnh vận tốc sang tín hiệu PWM (`cmdvel_to_pwm.py`), cũng như các tệp cấu hình YAML cho SLAM Toolbox, Navigation2 và AMCL. VS Code còn được dùng để chỉnh sửa tệp mô tả robot URDF/XACRO, launch file ROS2 và mã nguồn Arduino cho vi điều khiển Mega 2560.

Các tính năng của VS Code được khai thác trong quá trình phát triển bao gồm: tích hợp terminal cho phép chạy lệnh ROS2 trực tiếp trong cửa sổ làm việc, hỗ trợ làm nổi bật cú pháp cho Python, C++, YAML, XML và nhiều ngôn ngữ khác, tính năng IntelliSense giúp gợi ý mã và phát hiện lỗi cú pháp sớm, tích hợp Git để quản lý phiên bản mã nguồn, và hỗ trợ kết nối SSH từ xa giúp lập trình trực tiếp trên Raspberry Pi 4 từ máy tính phát triển mà không cần màn hình riêng.



Hình 2. 5 Giao diện Visual Studio Code

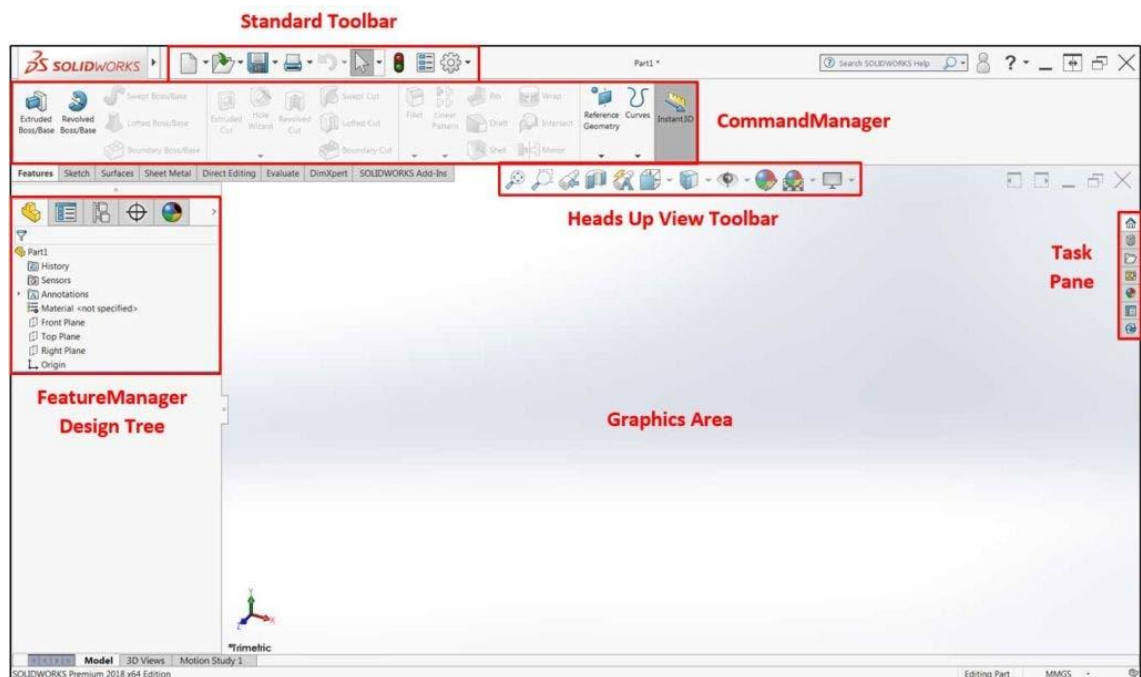
2.10.2 SolidWorks

SolidWorks là phần mềm thiết kế 3D tham số (parametric CAD) chuyên nghiệp do Dassault Systèmes phát triển, được sử dụng rộng rãi trong kỹ thuật cơ khí, tự động hóa và thiết kế sản phẩm. Phần mềm cho phép xây dựng các mô hình 3D chính xác từ các phác thảo 2D, lắp ráp nhiều chi tiết thành cụm lắp, kiểm tra va chạm và xuất bản vẽ kỹ thuật chuẩn ISO.

Trong đề tài, SolidWorks được sử dụng ở giai đoạn thiết kế cơ khí để tạo mô hình 3D toàn bộ khung xe robot, bao gồm tấm đế, khung đỡ linh kiện, vị trí lắp đặt bánh xe chủ động, bánh bị động, cơ cấu hai tầng để chứa vật phẩm và vị trí gắn cảm biến RPLidar A1M8 trên đỉnh. Mô hình này giúp nhóm kiểm tra bố cục linh kiện, xác định

khoảng cách giữa hai bánh chủ động — thông số đầu vào quan trọng cho công thức odometry bánh xe vi sai — và đánh giá tổng thể kích thước robot trước khi gia công và lắp ráp thực tế.

Sau khi mô hình SolidWorks được hoàn thiện, đề tài tiến hành xuất tệp theo định dạng tương thích để chuyển đổi sang URDF (Unified Robot Description Format) — định dạng mô tả robot tiêu chuẩn trong ROS2. Tệp URDF này được sử dụng để hiển thị mô hình 3D robot trong RViz2 và làm cơ sở cho các phép tính TF transform giữa các frame tọa độ của robot, đặc biệt là transform tĩnh giữa frame `base_link` và frame `laser_frame` của RPLidar A1M8.



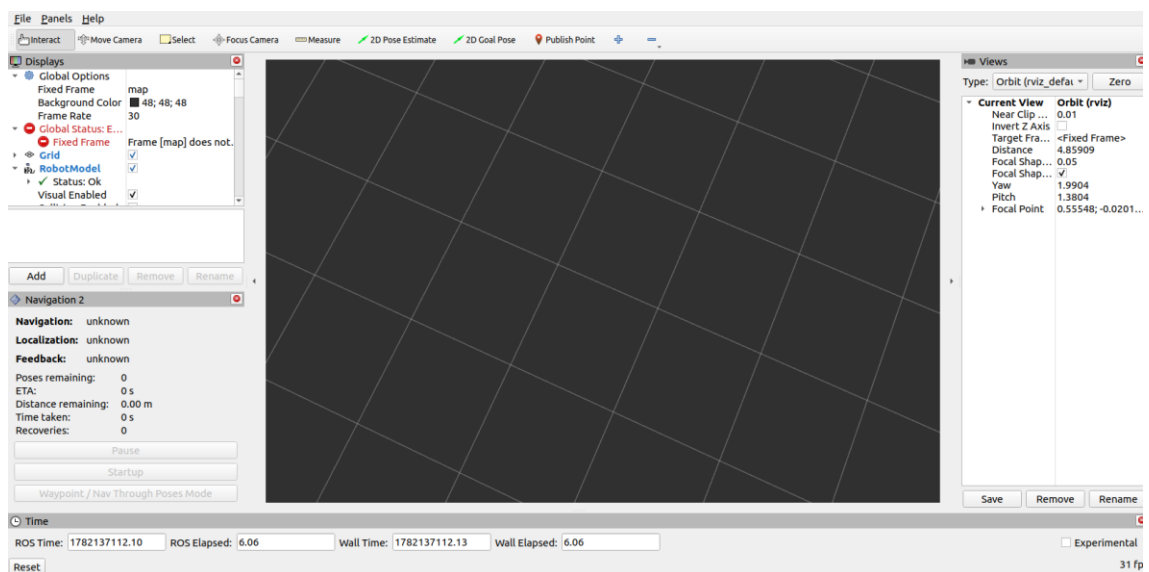
Hình 2. 6 Giao diện công cụ SolidWorks

2.10.3 Công cụ trực quan hóa RViz2

RViz2 là công cụ trực quan hóa 3D chính thức của ROS2, hoạt động theo cơ chế subscribe dữ liệu từ các topic và hiển thị đồng thời nhiều luồng thông tin trên cùng một giao diện không gian. Trong hệ thống của đề tài, RViz2 được sử dụng ở cả hai giai đoạn: giai đoạn SLAM để quan sát quá trình xây dựng bản đồ và giai đoạn Navigation2 để giám sát định vị, đường đi và vị trí robot.

Các thành phần được hiển thị trong RViz2 bao gồm bản đồ Occupancy Grid từ topic /map với màu trắng, đen và xám biểu thị ba trạng thái ô lưới; dữ liệu LaserScan từ topic /scan hiển thị dưới dạng tập hợp điểm xung quanh vị trí robot; vị trí và hướng robot hiển thị qua mô hình robot trong hệ tọa độ bản đồ; đám mây hạt AMCL (particle cloud) từ topic /particlecloud cho phép quan sát mức độ hội tụ định vị; đường đi toàn cục (global path) từ Planner Server; và các lớp costmap cục bộ và toàn cục từ Navigation2.

Ngoài chức năng hiển thị, RViz2 còn là công cụ tương tác trực tiếp với hệ thống: người dùng có thể cung cấp ước lượng vị trí ban đầu cho AMCL qua nút "2D Pose Estimate" và gửi Goal Pose cho Navigation2 qua nút "2D Nav Goal" trực tiếp trên giao diện bản đồ. Đây là hai thao tác bắt buộc trong mỗi phiên vận hành robot ở chế độ điều hướng tự động.



Hình 2. 7 Giao diện công cụ trực quan hóa RViz2

KẾT LUẬN CHƯƠNG 2

Chương 2 đã trình bày các cơ sở lý thuyết trực tiếp phục vụ cho đề tài robot phục vụ tự hành. Nội dung bao gồm tổng quan về AMR và robot phục vụ nhà hàng, nền tảng ROS2 Humble với các thành phần Node, Topic, TF và Action, công nghệ SLAM Toolbox phục vụ xây dựng bản đồ, hệ thống Navigation2 với Map Server, Planner

Server, Controller Server, BT Navigator và Costmap, phương pháp định vị AMCL, các thành phần phần cứng chính và công cụ RViz2. Những nội dung này là cơ sở để triển khai thiết kế phần cứng, phần mềm và quy trình thực nghiệm ở các chương tiếp theo.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Chương này trình bày quá trình thiết kế và xây dựng hệ thống robot phục vụ tự hành dựa trên các cơ sở lý thuyết đã nêu ở Chương 2. Nội dung chương gồm yêu cầu hệ thống, thiết kế tổng thể, thiết kế phần cứng robot, thiết kế phần mềm ROS2 và thiết kế website quản lý/tương tác.

3.1 Yêu cầu hệ thống

Bảng 3. 1 Tóm tắt yêu cầu hệ thống

STT	Yêu cầu thiết kế	Nội dung yêu cầu
1	Khả năng di chuyển	Robot có khả năng di chuyển linh hoạt trong môi trường trong nhà bằng cơ cấu dẫn động vi sai hai bánh chủ động.
2	Xây dựng bản đồ	Robot phải có khả năng quét môi trường và xây dựng bản đồ 2D theo thời gian thực bằng cảm biến LiDAR.
3	Định vị	Robot phải xác định được vị trí hiện tại trên bản đồ đã xây dựng với sai số chấp nhận được.
4	Điều hướng tự động	Robot có khả năng tự động di chuyển đến vị trí đích được chỉ định trên bản đồ.
5	Tránh vật cản	Robot phải phát hiện và xử lý vật cản xuất hiện trên đường di chuyển nhằm đảm bảo an toàn vận hành.
6	Điều khiển thủ công	Hỗ trợ điều khiển từ xa trong quá trình thử nghiệm, vận hành và xây dựng bản đồ.

7	Hiển thị trực quan	Dữ liệu bản đồ, vị trí robot, quỹ đạo và thông tin cảm biến phải được hiển thị trực quan trên RViz2.
8	Giao tiếp giữa các khối	Các thành phần phần cứng và phần mềm phải giao tiếp ổn định thông qua nền tảng ROS2.
9	Website quản lý	Hệ thống hỗ trợ giao diện website cho phép khách hàng lựa chọn bàn, gửi yêu cầu phục vụ và theo dõi trạng thái robot.
10	Khả năng mở rộng	Hệ thống có thể mở rộng thêm chức năng nhận diện vật thể, gọi món tự động hoặc nhiều robot trong tương lai.
11	Chi phí triển khai	Sử dụng các linh kiện phổ biến, chi phí phù hợp với mô hình nghiên cứu và đào tạo.
12	Độ ổn định	Robot hoạt động liên tục trong môi trường nhà hàng mô phỏng mà không xảy ra mất kết nối hoặc lỗi điều hướng nghiêm trọng.

Yêu cầu hệ thống được xác định dựa trên mục tiêu xây dựng một robot phục vụ tự hành trong môi trường nhà hàng mô phỏng. Các yêu cầu được chia thành yêu cầu chức năng và yêu cầu phi chức năng để thuận tiện cho quá trình thiết kế, triển khai và đánh giá.

Về chức năng robot, hệ thống cần thu nhận dữ liệu môi trường, tạo bản đồ bằng SLAM, định vị bằng AMCL và điều hướng đến các vị trí được chỉ định bằng Nav2. Về tương tác, website cần cung cấp giao diện để người dùng hoặc nhân viên tạo yêu cầu phục vụ, theo dõi trạng thái và quản lý thông tin bàn/món trong mô hình nhà hàng. Về phi chức năng, robot cần hoạt động ổn định trong không gian thử nghiệm, giao diện dễ thao tác và cấu trúc phần mềm đủ rõ ràng để có thể mở rộng.

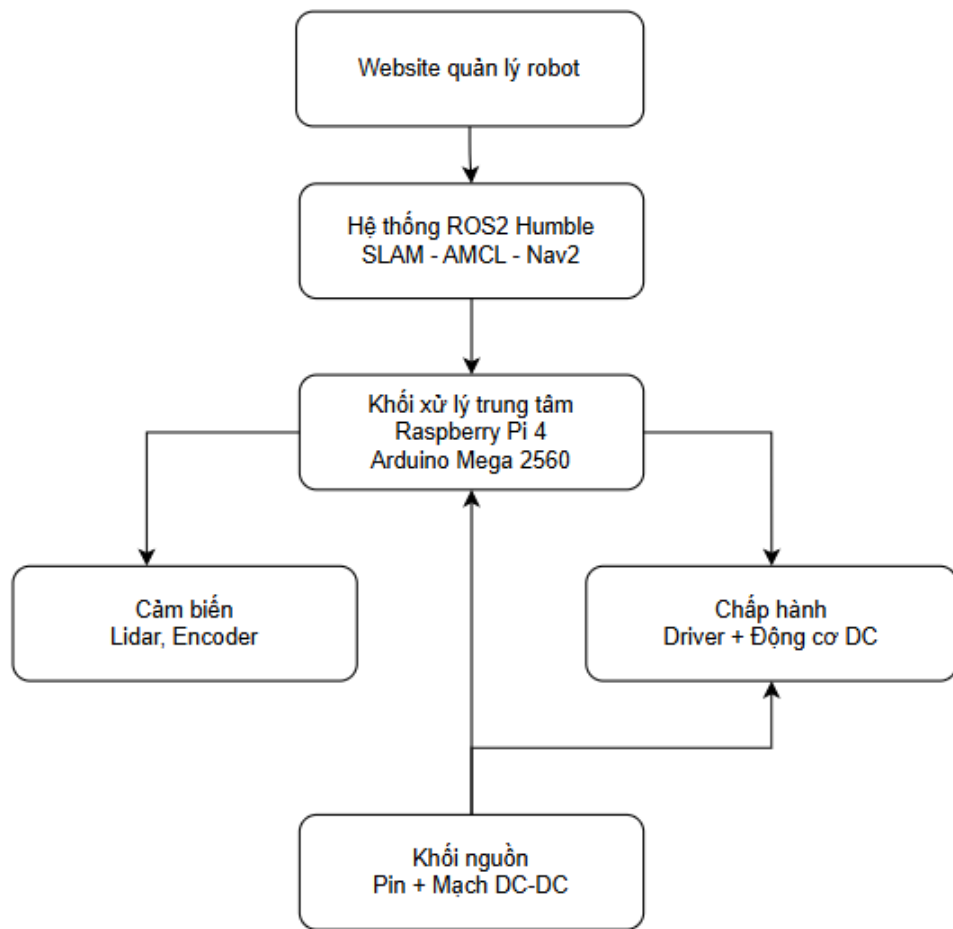
3.2 Thiết kế tổng thể hệ thống

Hệ thống được thiết kế theo mô hình phân tầng gồm ba lớp chính: lớp website/server, lớp xử lý robot trên ROS2 và lớp phần cứng robot. Lớp website/server

đảm nhiệm giao diện quản lý và theo dõi. Lớp ROS2 xử lý bản đồ, định vị, điều hướng và chuyển đổi lệnh vận tốc. Lớp phần cứng thực thi chuyển động và thu thập dữ liệu cảm biến.

Luồng dữ liệu tổng thể bắt đầu từ yêu cầu của khách hàng hoặc người quản lý trên website. Yêu cầu được gửi đến server, server xác định nhiệm vụ và truyền thông tin cần thiết sang lớp ROS2. ROS2 dùng bản đồ, vị trí robot và Nav2 để sinh lệnh điều hướng. Lệnh /cmd_vel được node điều khiển chuyển thành dữ liệu PWM gửi xuống Arduino. Arduino điều khiển BTS7960 để dẫn động hai động cơ, đồng thời đọc encoder gửi ngược về Raspberry Pi 4.

Ở chiều cảm biến, RPLidar A1 liên tục gửi dữ liệu quét lên Raspberry Pi 4. Encoder gửi thông tin chuyển động qua Arduino. Hai nguồn dữ liệu này được sử dụng cho SLAM, AMCL và RViz2. Cách tổ chức này giúp hệ thống tách biệt rõ giữa giao diện ứng dụng, phần mềm robot và phần cứng điều khiển.



Hình 3. 1 Sơ đồ khối tổng thể hệ thống

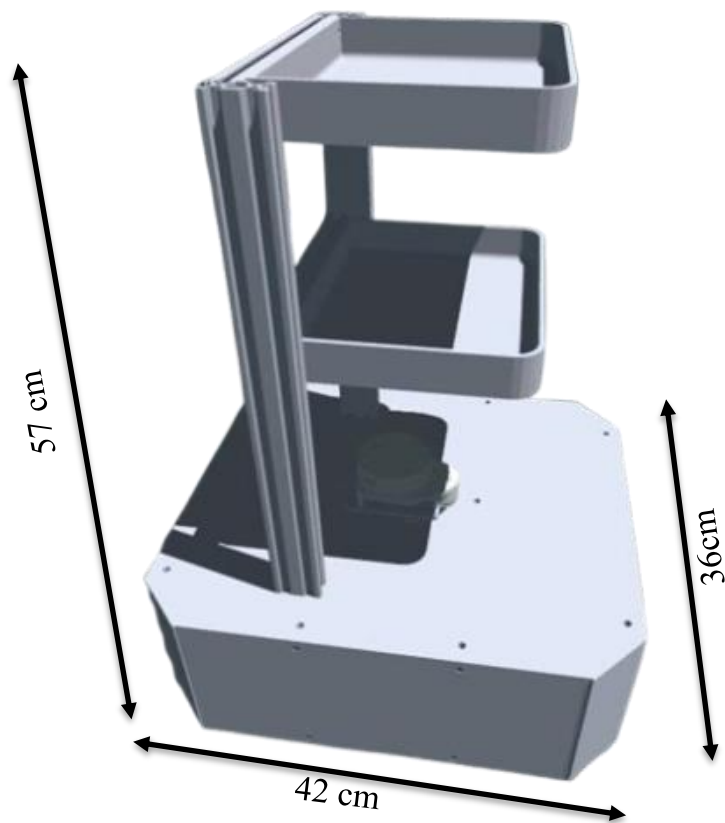
3.3 Thiết kế phần cứng robot

Phần cứng robot được thiết kế theo cấu trúc robot di động hai bánh chủ động. Các khối chính gồm khung xe, động cơ DC encoder, Arduino Mega 2560, Raspberry Pi 4, RPLidar A1, driver BTS7960 và khối nguồn. Việc lựa chọn các thành phần này dựa trên yêu cầu chi phí, khả năng tích hợp và mức độ hỗ trợ trong ROS2.

3.3.1 Thi công lắp ráp mô hình Robot

Khung là nền tảng cơ khí để lắp đặt các thành phần phần cứng. Robot sử dụng cấu trúc bánh xe vi sai gồm hai bánh chủ động hai bên và bánh tự do hỗ trợ cân bằng. Cấu trúc này có ưu điểm là đơn giản, dễ điều khiển và phù hợp với môi trường trong nhà. Trên khung xe, LiDAR cần được đặt ở vị trí cao và ít bị che khuất để có thể quét 360

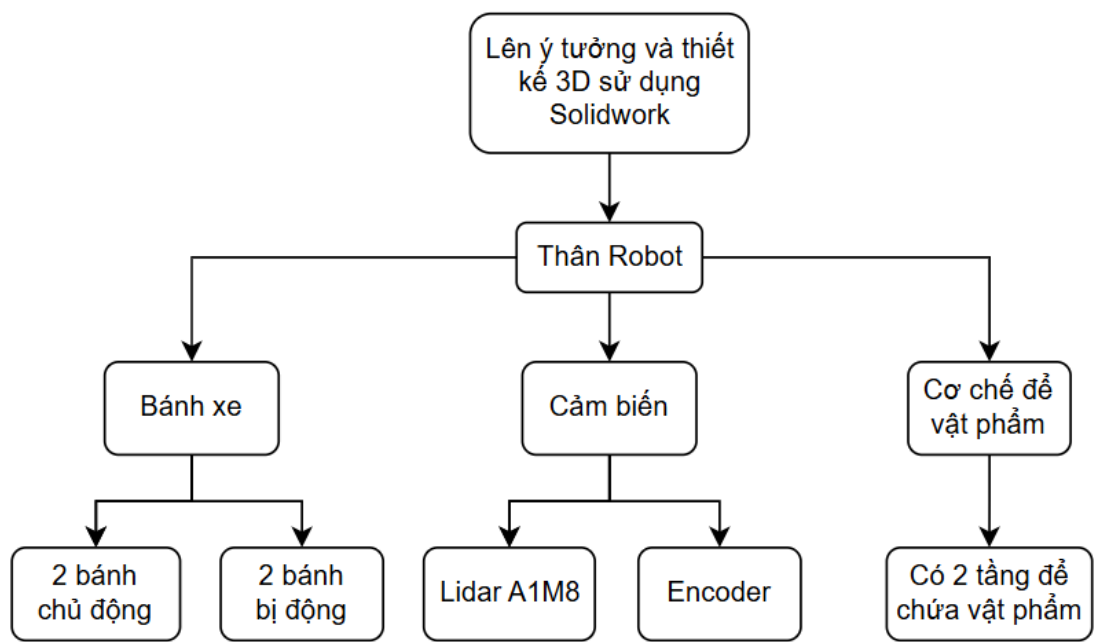
độ. Raspberry Pi 4, Arduino, driver và khối nguồn được bố trí sao cho dây dẫn gọn, hạn chế nhiễu và dễ bảo trì.



Hình 3. 2 Mô phỏng 3D Robot trên Solidworks



Hình 3. 3 Mô hình Robot thực tế và vị trí lắp đặt linh kiện



Hình 3. 4 Sơ đồ cấu trúc thân Robot

Cấu trúc tổng thể của xe được xây dựng dựa trên quá trình lên ý tưởng và mô hình hóa 3D bằng phần mềm SolidWorks. Việc thiết kế này giúp đảm bảo độ chính xác về cơ khí, thuận tiện trong quá trình lắp ráp, đồng thời tối ưu không gian bố trí các bộ phận bên trong xe. Tổng thể hệ thống được chia thành các cụm chức năng chính gồm thân xe, hệ thống bánh xe, hệ thống cảm biến và cơ cấu để chứa vật phẩm.

Thân xe là bộ phận đóng vai trò như khung chịu lực chính của toàn bộ hệ thống. Đây là nơi liên kết, gá lắp và cố định các cụm chi tiết khác trên xe. Thiết kế thân xe được tính toán nhằm đảm bảo độ cứng vững trong quá trình di chuyển, đồng thời tạo đủ không gian để bố trí cảm biến, cơ cấu để chứa vật phẩm và các linh kiện liên quan. Bên cạnh đó, kết cấu thân xe cũng được thiết kế theo hướng đơn giản, thuận tiện cho việc gia công, lắp ráp và bảo trì khi cần thiết.

Hệ thống bánh xe được bố trí nhằm giúp xe di chuyển linh hoạt, ổn định và phù hợp với yêu cầu vận hành. Trong đó, hai bánh chủ động đảm nhận nhiệm vụ tạo lực kéo và điều khiển hướng chuyển động của xe. Hai bánh bị động có chức năng hỗ trợ cân bằng, phân bố tải trọng và giúp xe duy trì sự ổn định trong quá trình di chuyển. Cách bố trí này góp phần nâng cao khả năng vận hành của xe trên nhiều dạng bề mặt khác nhau.

Hệ thống cảm biến được tích hợp trên xe nhằm thu thập dữ liệu phục vụ cho quá trình điều khiển, định vị và dẫn hướng chuyển động. Cảm biến LiDAR A1M8 được sử dụng để quét môi trường xung quanh, phát hiện vật cản và hỗ trợ xe di chuyển an toàn. Bên cạnh đó, encoder được lắp đặt để đo tốc độ quay của bánh xe và xác định quãng đường di chuyển, từ đó giúp hệ thống điều khiển vận tốc và vị trí của xe một cách chính xác hơn.

Cơ cấu để chứa vật phẩm được thiết kế theo dạng hai tầng nhằm tăng khả năng lưu trữ và hỗ trợ phân loại vật phẩm một cách thuận tiện. Thiết kế nhiều tầng giúp tận dụng hiệu quả không gian theo phương thẳng đứng của thân xe, đồng thời vẫn đảm bảo sự gọn gàng và hợp lý trong bố trí tổng thể. Nhờ đó, xe có thể mang được nhiều vật phẩm hơn mà không làm tăng đáng kể diện tích chiếm chỗ.

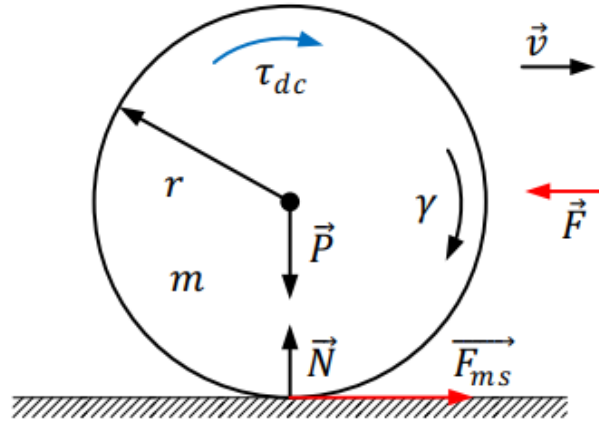
3.3.2 Động cơ Dc encoder JGB37-520 và Driver BTS7960

3.3.2.1 Động cơ Dc encoder JGB37-520

Robot sử dụng hai động cơ DC encoder JGB37-520 để tạo chuyển động. Mỗi động cơ được gắn với một bánh chủ động, cho phép robot tiến, lùi, quay trái, quay phải hoặc xoay tại chỗ bằng cách thay đổi tốc độ tương đối giữa hai bánh. Encoder tích hợp trong động cơ cung cấp xung phản hồi để tính toán vận tốc và quãng đường di chuyển.

Thông tin encoder được Arduino Mega 2560 đọc liên tục và gửi lên Raspberry Pi 4 theo frame dạng E,<countL>,<countR>. Từ số xung bánh trái và bánh phải, node odometry trên Raspberry Pi tính toán vị trí tương đối của robot, xuất bản topic /odom và phát TF odom → base_link. Đây là dữ liệu quan trọng cho SLAM, AMCL và Nav2.

-Lí do chọn động cơ DC JGB37-520:



Hình 3. 5 Hình 3.5. Mô hình phân tích lực một bánh xe

Moment quán tính I của bánh xe là thành phần khó xác định nên thay vào đó xem bánh xe như đĩa tròn đồng chất và tính gần đúng bằng công thức (2.6):

$$I = \frac{1}{2}mr^2 = \frac{1}{2} \times 0.05 \times 0.049^2 = 6 \times 10^{-5}$$

Gia tốc góc:

$$\gamma = \frac{a}{r} = \frac{0.5}{0.049} = \frac{500}{49} \text{ (rad/s)}$$

Áp dụng định luật II Newton theo phương ngang, lực tác dụng lên thành phần dẫn động có thể xác định như sau:

$$f = \left(\frac{1}{2}M + m \right) a = \left(\frac{1}{2} \times 4 + 0.05 \right) \times 0.05 = 1.025 \text{ N}$$

Momen xoắn tác dụng lên bánh xe (giả sử bánh xe lăn không trượt)

$$\begin{aligned} \tau_{dc} - Fr &= I\gamma \\ \Rightarrow \tau_{dc} &= Fr + I\gamma = 1.025 \times 0.049 + 6 \times 10^{-5} \times \frac{500}{49} \end{aligned}$$

$$= 0.05 Nm$$

Điều kiện để bánh lăn không trượt:

$$F_{ms} \geq F$$

$$\Leftrightarrow \mu g = \frac{1}{2} M + m \geq 1.025$$

$$\Leftrightarrow \mu \geq \frac{1.025}{9.81 \times \left(\frac{1}{2} \times 4 + 0.05 \right)} = 0.051$$

Lớp nhựa cứng có răng thường có hệ số ma sát lớn hơn 0.5 để giữ bánh lăn không trượt. Bánh xe được dẫn động thông qua khớp nối lực góc với hiệu suất là $\eta_{kn} < 0,98$ (theo tài liệu [9]). Do đó, momen xoắn cần thiết mà động cơ cần phải cung cấp là:

$$\tau_{ct} = \frac{\tau_{dc}}{0.98} = \frac{Fr}{0.98} = \frac{1.025 \times 0.049}{0.98} = 0.051 Nm$$

Số vòng quay yêu cầu của bánh xe:

$$n_{tt} = \frac{60v_{max}}{2\pi r} = \frac{60 \times 0.3}{2\pi \times 0.049} = 58.5 \text{ vòng/phút}$$

Công suất của mỗi động cơ:

$$P = \frac{k \times \tau_{ct} \times n_{tt}}{9.55}$$

Trong đó: k là hệ số an toàn, chọn $k = 1.5$

$$p = \frac{1.5 \times 0.051 \times 58.6}{9.55} = 0.46 \text{ W}$$

Dựa trên các thông số động cơ được trình bày trong bảng. , động cơ phù hợp nhất sẽ được lựa chọn nhằm đáp ứng các yêu cầu đặt ra:

$$P_{dc} \geq P_{tt} = 0.46 \text{ W}$$

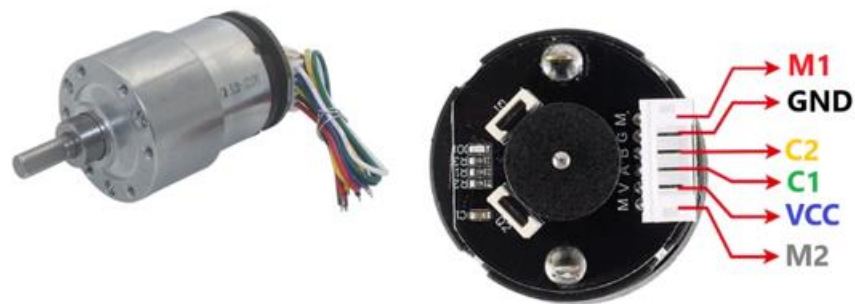
$$\text{Và } n_{dc} \geq n_{tt} = 58.6 \text{ vòng/phút}$$

Để tìm động cơ để phù hợp với yêu cầu hệ thống và đồng thời phù hợp với giá thành. Do đó, động cơ DC JGB37-520 là lựa chọn phù hợp nhờ giá thành vừa phải. Ngoài ra, với kích thước nhỏ động cơ này thích hợp để lắp đặt bên trong robot, giúp thiết kế robot trở nên gọn gàng và linh hoạt hơn.

Bảng 3. 2 Thông số các loại động cơ dẫn động:

Thông số	JGB37-520	JGA25 - 370	Planetary GP36
Điện áp sử dụng	12 VDC	12 VDC	12 VDC
Công suất định mức	12W	21 W	6 W (max 23W)
Tốc độ vòng quay	60 rpm	280 rpm	145 rpm
Lực kéo moment định mức	30 kg.cm	2 kg.cm	12.2 kg.cm
Tỉ số truyền bộ giảm tốc	168:1	21.3:1	1 :27
Giá thành	245.000 VNĐ	165.000 VNĐ	1.112.000VN Đ
Chiều dài hộp số	26.5 mm	19 mm	39 mm

Bảng 3. 2



Hình 3. 6 Động cơ DC Encoder JGB37-520

3.3.2.2 Driver BTS7960

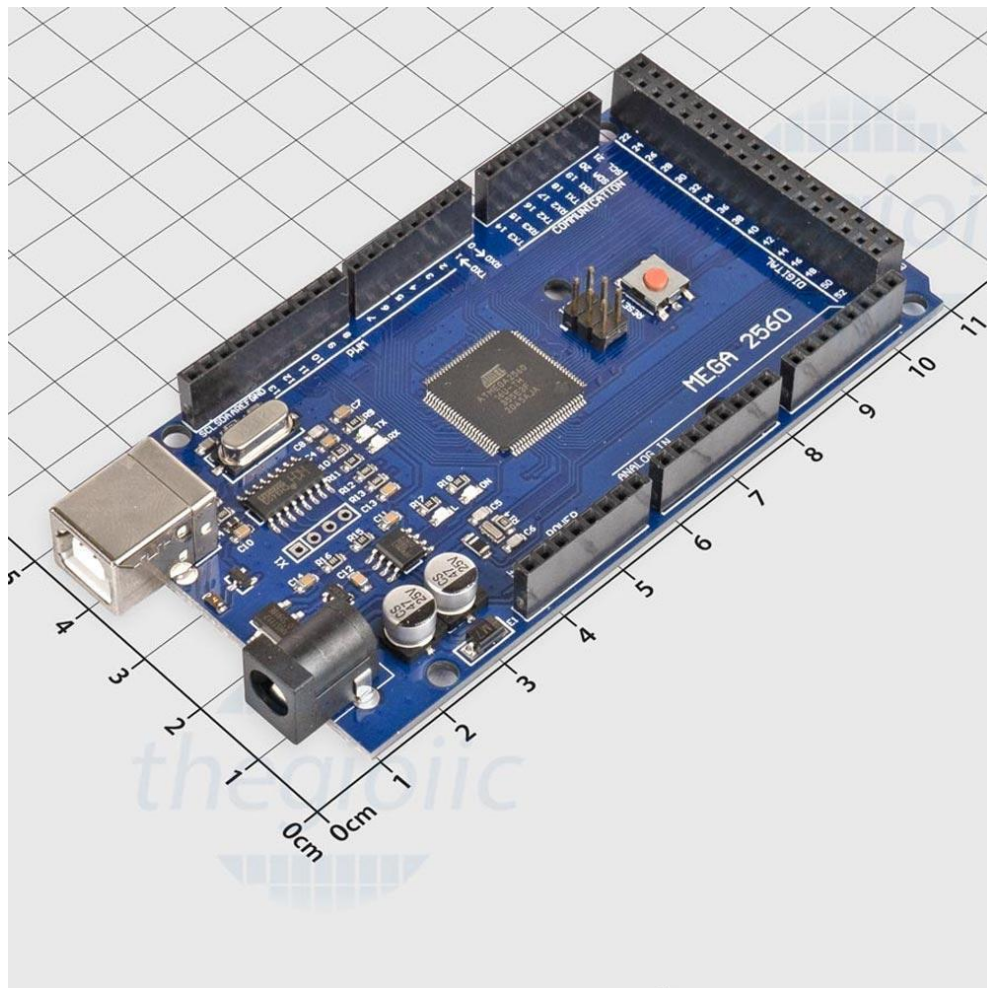
Driver BTS7960 là module điều khiển công suất sử dụng cấu trúc cầu H toàn phần, cho phép điều khiển chiều quay và tốc độ động cơ thông qua tín hiệu PWM từ Arduino. Mỗi module BTS7960 điều khiển một động cơ độc lập và tích hợp bảo vệ quá dòng và quá nhiệt. Mạch hạ áp XL4016 là DC-DC Buck Converter có hiệu suất cao, nhận điện áp từ pin 18650 và ổn định đầu ra ở mức phù hợp cho từng thành phần trong hệ thống, phân phối theo hai nhánh độc lập để tách biệt nhiễu điện từ của động cơ khỏi mạch điều khiển.[10]



Hình 3. 7 Driver BTS7960

3.3.3 Arduino Mega 2560

Arduino Mega 2560 là vi điều khiển dựa trên chip ATmega2560, đảm nhiệm vai trò bộ điều khiển cấp thấp trong kiến trúc phân tầng của hệ thống. Arduino thực hiện hai nhiệm vụ chính theo thời gian thực: đọc tín hiệu xung từ encoder của hai động cơ qua ngắt ngoài và gửi dữ liệu thô về Raspberry Pi 4; đồng thời nhận lệnh vận tốc từ Raspberry Pi 4 và xuất tín hiệu PWM tương ứng cho hai driver BTS7960 điều khiển động cơ. Sự phân tách giữa Arduino và Raspberry Pi 4 đảm bảo các tác vụ thời gian thực của điều khiển động cơ không bị ảnh hưởng bởi tải xử lý của các thuật toán SLAM và Nav2.[11]

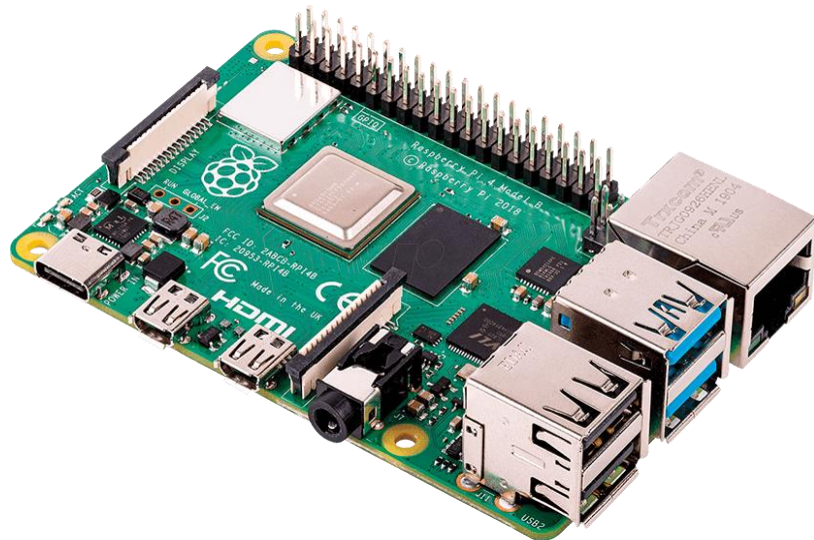


Hình 3. 8 Arduino Mega 2560

Thông số	Arduino Mega 2560
Bộ xử lý	ATmega2560
Tốc độ xung nhịp	16 MHz
Số chân Digital I/O	54
Số chân Analog	16
Chuẩn giao tiếp	UART, SPI, I2C
Điện áp vào khuyến nghị	7 ÷ 12 V
Điện áp hoạt động	5 V

3.3.4 Raspberry Pi 4 Mode B

Raspberry Pi 4 là máy tính nhúng đơn board với bộ xử lý ARM Cortex-A72 bốn nhân, đóng vai trò bộ xử lý trung tâm cấp cao của hệ thống. Thiết bị chạy Ubuntu 22.04 và toàn bộ stack phần mềm ROS2 Humble bao gồm driver RPLidar A1, Node giao tiếp Serial, SLAM Toolbox, Navigation2 với AMCL và RViz2. Raspberry Pi 4 kết nối với RPLidar A1 qua USB và kết nối với Arduino Mega 2560 qua USB Serial ở tốc độ 115200 baud, đồng thời kết nối mạng để giao tiếp với hệ thống website.[12]



Hình 3. 9 Raspberry Pi 4

Bảng 3. 3 Thông số Raspberry Pi 4 Mode B

Thông số	Raspberry Pi 4 Model B
Bộ xử lý	Broadcom BCM2711, Quad-core Cortex-A72 (64-bit)
Tốc độ xung nhịp	1.5 GHz

Bộ nhớ RAM	8 GB
Bộ nhớ lưu trữ	Thẻ nhớ microSD
Chuẩn giao tiếp	UART, SPI, I2C, USB, Ethernet, Wi-Fi, Bluetooth
Điện áp cấp nguồn	5 V DC
Dòng điện khuyến nghị	3 A
Số chân GPIO	40 chân
Cổng hiển thị	2 × Micro HDMI
Hệ điều hành hỗ trợ	Raspberry Pi OS, Ubuntu, ROS2

3.3.5 RPLidar A1

LiDAR (Light Detection and Ranging) là công nghệ đo khoảng cách bằng tia laser, hoạt động theo nguyên lý đo thời gian bay (Time of Flight): thiết bị phát tia laser và đo khoảng thời gian từ lúc phát đến khi nhận tín hiệu phản xạ, tính khoảng cách theo công thức $d = (c \times t)/2$. RPLidar A1M81 là thiết bị LiDAR 2D quét 360° của Slamtec, sử dụng nguyên lý đo quang học tam giác kết hợp cơ cấu quay liên tục để thu thập dữ liệu khoảng cách toàn hướng. Thiết bị có tần số quét khoảng 5,5 Hz, lấy mẫu lên đến 8.000 điểm mỗi giây và tầm đo tối đa 12 mét, đủ để bao phủ các môi trường trong nhà thông thường. [13]

RPLidar A1M81 đóng hai vai trò trong hệ thống của đề tài. Ở giai đoạn SLAM, dữ liệu LaserScan từ RPLidar A1 là nguồn đầu vào chính để SLAM Toolbox xây dựng bản đồ và định vị robot. Ở giai đoạn Navigation2, dữ liệu này được AMCL sử dụng để định vị robot trên bản đồ đã lưu, đồng thời cung cấp cho local costmap để

phát hiện và tránh vật cản động trong thời gian thực.



Hình 3. 10 Cảm biến LiDAR RPLidar A1

Bảng 3. 4 Thông số cảm biến LiDAR RPLidar A1M81

Thông số	RPLIDAR A1M8-R1
Loại cảm biến	LiDAR quét laser 2D
Phương thức đo	Tam giác lượng giác (Triangulation)
Phạm vi đo	0.15 – 12 m
Tần số quét	5.5 Hz
Tốc độ lấy mẫu	8000 mẫu/giây
Góc quét	360°
Độ phân giải góc	< 1°
Sai số đo khoảng cách	±1%
Nguồn cấp	5 V DC
Dòng điện hoạt động	Khoảng 400 mA
Chuẩn giao tiếp	UART (Serial)
Tốc độ truyền dữ liệu	115200 bps
Kích thước	96.8 × 70.3 mm

3.3.6 Khối nguồn

Khối nguồn có nhiệm vụ cung cấp năng lượng cho toàn bộ hệ thống robot, bao gồm Raspberry Pi 4, Arduino Mega 2560, mạch điều khiển động cơ BTS7960, cảm

biến LiDAR và các thiết bị ngoại vi khác. Nguồn điện của hệ thống được thiết kế theo hướng di động nhằm đảm bảo robot có thể hoạt động độc lập mà không phụ thuộc vào nguồn điện bên ngoài.

Nguồn chính của robot được tạo thành từ bốn pin lithium-ion 18650 có điện áp danh định 3,7 V, dung lượng 2600 mAh và năng lượng 9,49 Wh mỗi pin. Các pin được mắc nối tiếp để tạo thành bộ nguồn có điện áp giao động $3,7 \times 4 = 14,8V$, đáp ứng yêu cầu cấp nguồn cho hệ thống động lực và các khối xử lý trên robot.

Để cung cấp mức điện áp phù hợp cho từng thiết bị, hệ thống sử dụng hai mạch hạ áp DC-DC XL4016. Mạch hạ áp thứ nhất được điều chỉnh đầu ra để cấp nguồn cho Raspberry Pi 4 và Arduino Mega2560, đảm bảo điện áp ổn định cho quá trình xử lý dữ liệu và vận hành hệ thống. Mạch hạ áp thứ hai được sử dụng để tạo nguồn cấp cho mạch điều khiển động cơ BTS7960, giúp cung cấp năng lượng ổn định cho hệ thống truyền động của robot.

Việc sử dụng các bộ chuyển đổi hạ áp XL4016 giúp nâng cao hiệu suất chuyển đổi năng lượng, giảm tổn hao công suất và hạn chế hiện tượng sụt áp khi hệ thống hoạt động với tải lớn. Đồng thời, phương án thiết kế này còn tạo điều kiện thuận lợi cho việc mở rộng hoặc nâng cấp hệ thống nguồn trong các phiên bản phát triển tiếp theo.



Hình 3. 11 Mạch nguồn XL4016

Bảng 3. 5 Thông số DC-DC XL4016

Thông số	DC-DC XL4016
IC điều khiển	XL4016E1
Loại mạch	Bộ chuyển đổi giảm áp (Buck Converter)
Điện áp đầu vào	8 ÷ 40 V DC
Điện áp đầu ra	1.25 ÷ 36 V DC (điều chỉnh được)
Công suất đầu ra tối đa	200 W
Tần số chuyển mạch	180 kHz
Dòng điện đầu ra cực đại	10 A
Hiệu suất chuyển đổi	Lên đến 94%

3.4 Thiết kế phần mềm robot

Phần mềm robot được xây dựng trên ROS2 Humble theo kiến trúc nhiều node. Mỗi node đảm nhiệm một chức năng cụ thể và trao đổi dữ liệu qua topic, TF, service hoặc action. Cấu trúc này giúp hệ thống dễ kiểm tra từng phần và dễ bổ sung các chức năng như SLAM, AMCL hoặc Nav2.

3.4.1 Kiến trúc ROS2

Kiến trúc ROS2 của robot gồm các luồng chính: luồng LiDAR, luồng encoder/odometry, luồng điều hướng và luồng điều khiển động cơ. Node RPLidar đọc dữ liệu cảm biến và xuất bản /scan. Node encoder_odometry đọc dữ liệu encoder từ Serial, tính odometry và xuất bản /odom cùng TF odom → base_link. SLAM Toolbox hoặc AMCL sử dụng /scan, /odom và TF để tạo bản đồ hoặc định vị.

Ở phía điều khiển, Nav2 tạo lệnh vận tốc trên /cmd_vel. Node cmdvel_to_pwm_serial nhận /cmd_vel, chuyển vận tốc tuyến tính và vận tốc góc thành tốc độ bánh trái/phải, ánh xạ sang PWM và gửi xuống Arduino theo frame M,<pwmL>,<pwmR>. Node này cũng có cơ chế dừng an toàn khi không nhận lệnh /cmd_vel trong một khoảng thời gian, giúp robot không tiếp tục chạy khi mất tín hiệu điều khiển.

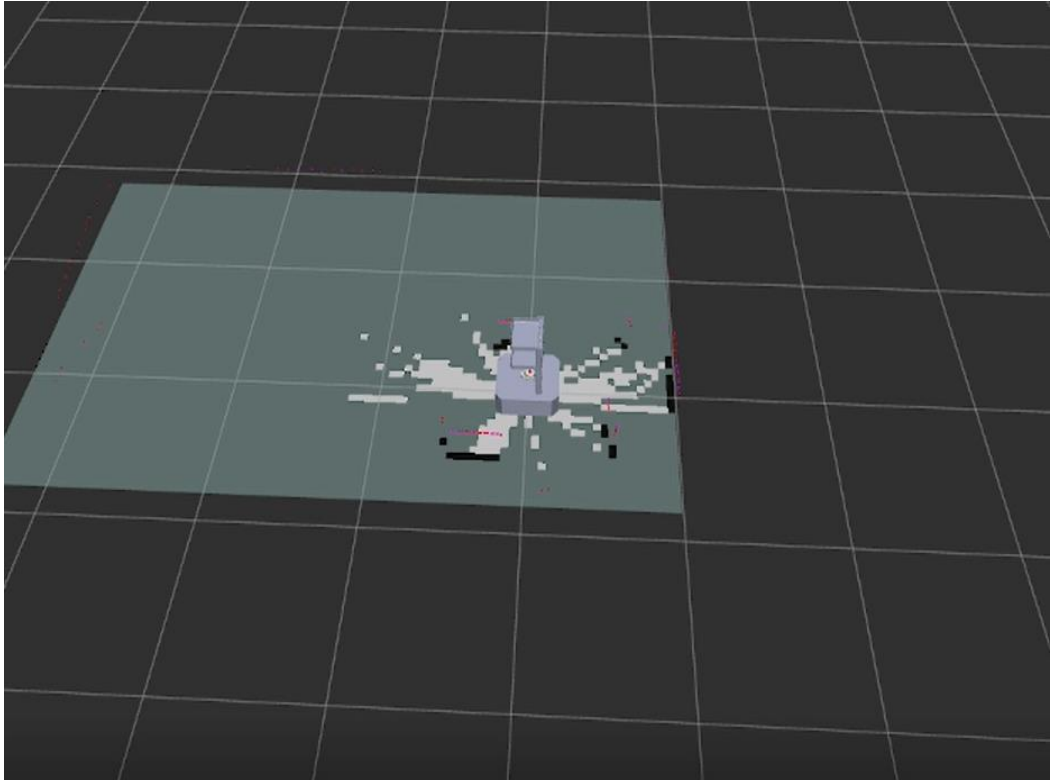
Bảng 3. 6 Các node chính trong kiến trúc phần mềm

Node	Dữ liệu vào	Dữ liệu ra
rplidar_node	USB LiDAR	/scan
encoder_odometry	Serial encoder	/odom, /tf
slam_toolbox	/scan, /odom, /tf	/map
amcl	/map, /scan, /odom	pose
nav2	goal, map, pose	/cmd_vel
cmdvel_to_pwm_serial	/cmd_vel	Serial PWM

3.4.2 Xây dựng bản đồ bằng SLAM

Quy trình xây dựng bản đồ bắt đầu bằng việc khởi động RPLidar A1, node odometry, SLAM Toolbox và RViz2. Người vận hành điều khiển robot di chuyển quanh môi trường để LiDAR quét đầy đủ các vùng cần lập bản đồ. SLAM Toolbox sử dụng dữ liệu /scan kết hợp với odometry và TF để cập nhật bản đồ Occupancy Grid theo thời gian thực.

Khi bản đồ đã đủ thông tin, người dùng lưu bản đồ thành tệp .pgm và .yaml. Tệp .pgm lưu dữ liệu ảnh bản đồ, còn tệp .yaml lưu các thông số như độ phân giải, gốc tọa độ, ngưỡng occupied/free. Bản đồ này được sử dụng lại trong bước định vị bằng AMCL và điều hướng bằng Nav2.



Hình 3. 12 Robot khi bắt đầu chạy quét map trên Rviz2



Hình 3. 13 Robot đã quét Map xong trong Rviz2



Hình 3. 14 Robot chạy quét Map trên thực tế

3.4.3 Định vị bằng AMCL

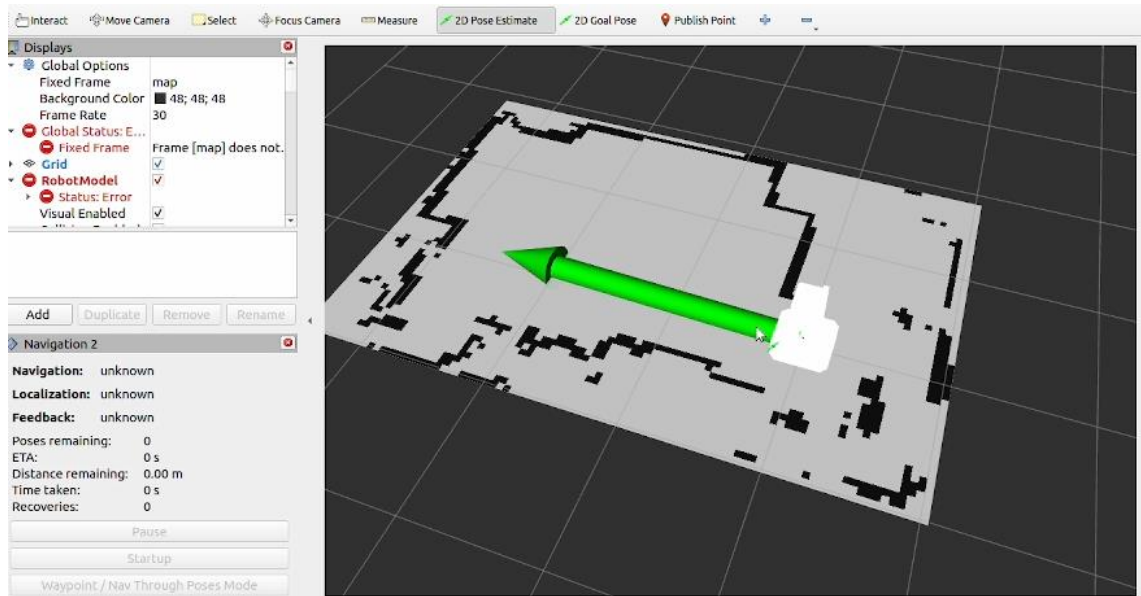
Sau khi có bản đồ, hệ thống chuyển sang chế độ định vị. Map Server nạp bản đồ đã lưu, AMCL nhận dữ liệu /scan, /odom và TF để ước lượng vị trí robot trên bản đồ. Khi khởi động, người vận hành có thể đặt vị trí ban đầu cho robot trong RViz2 bằng công cụ 2D Pose Estimate. Sau đó AMCL cập nhật vị trí dựa trên chuyển động và dữ liệu LiDAR mới.

Kết quả định vị của AMCL là đầu vào trực tiếp cho Nav2. Nếu vị trí ước lượng bị sai, planner có thể tạo đường đi không phù hợp hoặc robot không bám đúng đường. Vì vậy, bước kiểm tra AMCL trên RViz2 là cần thiết trước khi thử nghiệm điều hướng.

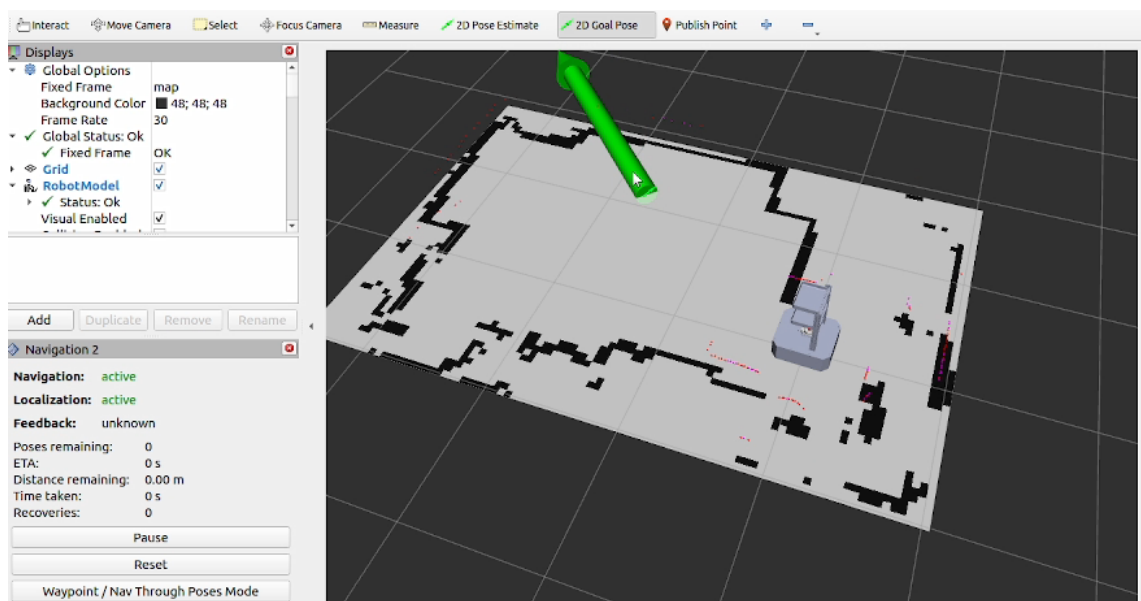
3.4.4 Điều hướng bằng Navigation

Trong chế độ điều hướng, người dùng đặt Goal Pose trên RViz2 hoặc gửi mục tiêu từ hệ thống quản lý. Nav2 tiếp nhận mục tiêu, dùng Planner Server để tạo đường đi từ vị trí hiện tại đến vị trí đích, sau đó Controller Server tạo lệnh vận tốc /cmd_vel để robot bám theo đường đi. Các costmap cục bộ và toàn cục được sử dụng để biểu diễn vùng vật cản và vùng có thể di chuyển.

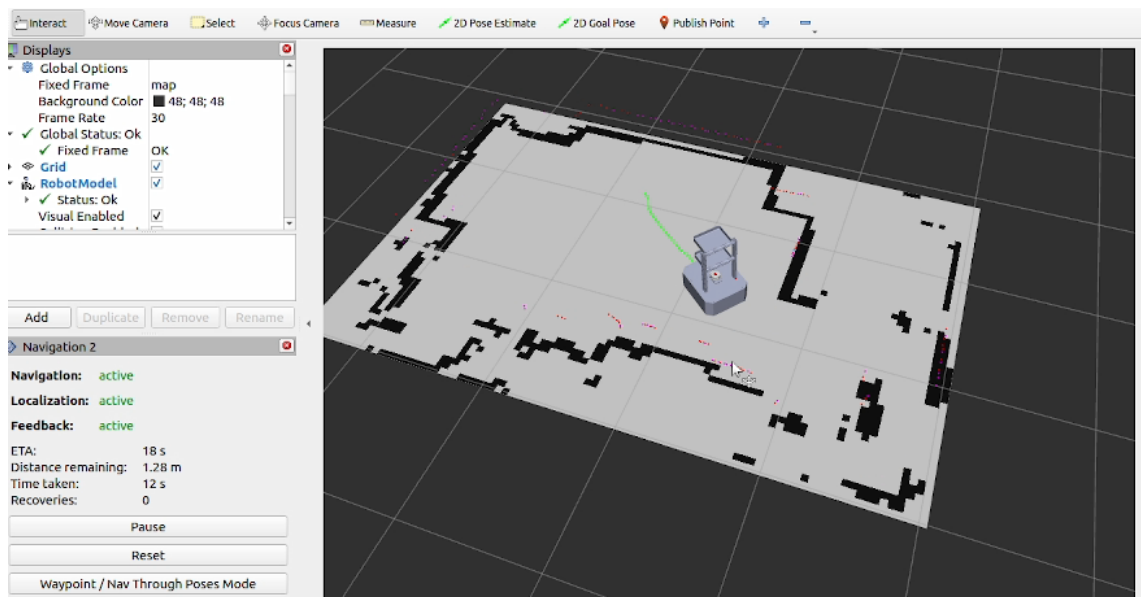
Lệnh `/cmd_vel` do Nav2 tạo ra không trực tiếp điều khiển động cơ mà được node `cmdvel_to_pwm_serial` chuyển đổi. Node này sử dụng thông số bán kính bánh xe và khoảng cách hai bánh để tính vận tốc bánh trái/phải, sau đó đổi sang PWM gửi xuống Arduino. Nhờ vậy, Nav2 có thể điều khiển robot thông qua cùng một giao diện vận tốc chuẩn của ROS2.



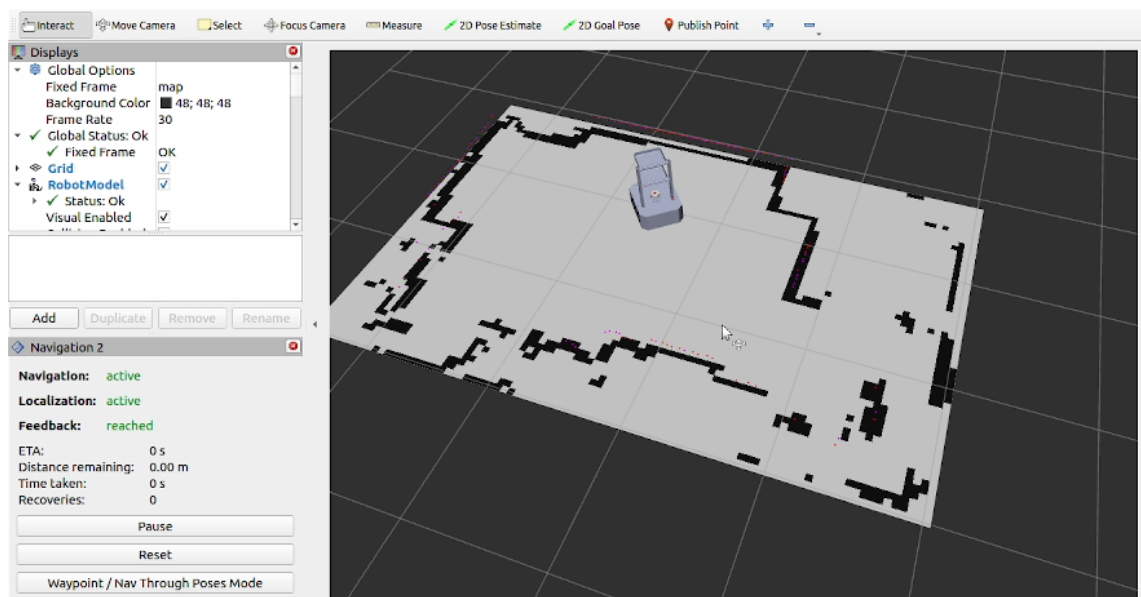
Hình 3. 15 Thiết lập hướng đi và vị trí ban đầu của robot



Hình 3. 16 Dùng Goal Pose để thiết lập hướng và vị trí đến



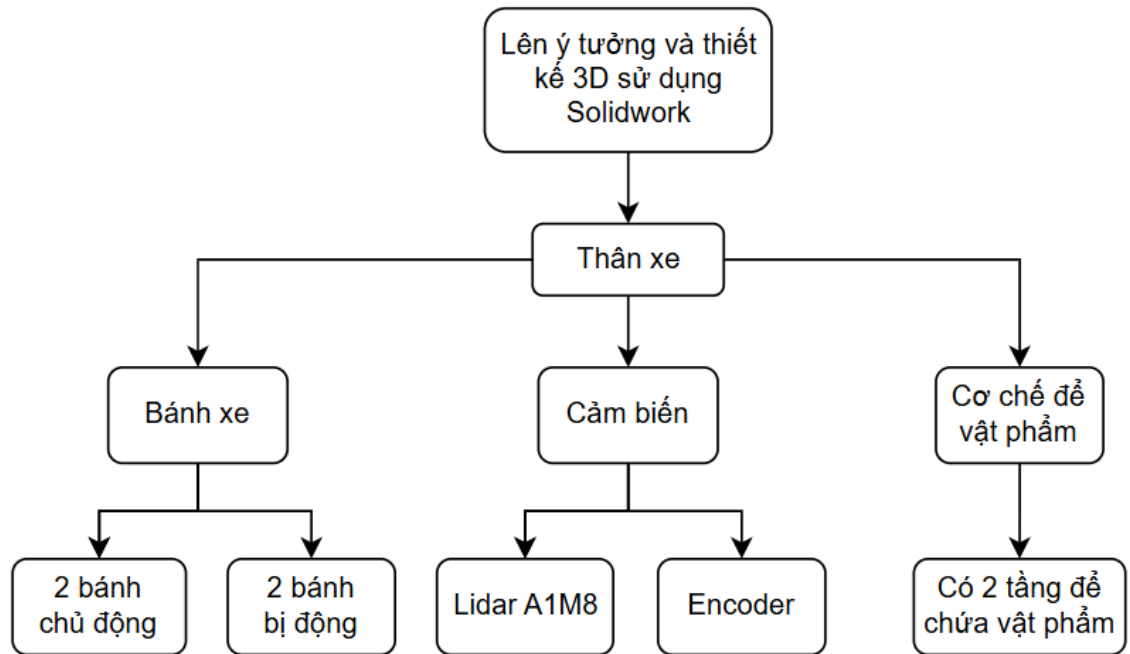
Hình 3. 17 Quá trình di chuyển của robot đến vị trí chỉ định trên Rviz2



Hình 3. 18 Robot đã di chuyển đến vị trí được chỉ định Trên Rviz2

3.5. Sơ đồ và lưu đồ hoạt động của hệ thống

3.5.1 Sơ đồ cấu trúc của robot



Hình 3. 19 sơ đồ cấu trúc thân Robot

Cấu trúc của xe được thiết kế dựa trên quá trình lên ý tưởng và mô hình hóa 3D bằng phần mềm SolidWorks, nhằm đảm bảo tính chính xác về mặt cơ khí, khả năng lắp ráp và tối ưu không gian bố trí các bộ phận. Tổng thể xe bao gồm các khối chức năng chính như sau:

-Thân xe

Thân xe đóng vai trò là khung chịu lực chính, là nơi liên kết và cố định toàn bộ các bộ phận còn lại của hệ thống. Thiết kế thân xe đảm bảo:

- Độ cứng vững khi xe di chuyển
- Không gian phù hợp để lắp đặt cảm biến và cơ cấu để thuốc
- Dễ dàng gia công và bảo trì

- Hệ thống bánh xe

Hệ thống bánh xe được bố trí nhằm đảm bảo khả năng di chuyển linh hoạt và ổn định cho xe, bao gồm:

- 2 bánh chủ động: chịu trách nhiệm tạo lực kéo và điều khiển chuyển động của xe

- 2 bánh bị động: có vai trò giữ thăng bằng, hỗ trợ phân bố tải trọng và ổn định hướng di chuyển

Cách bố trí này giúp xe vận hành ổn định trên nhiều bề mặt khác nhau.

-Hệ thống cảm biến

Hệ thống cảm biến được tích hợp nhằm thu thập dữ liệu phục vụ cho việc điều khiển và định hướng chuyển động của xe, bao gồm:

- Cảm biến LiDAR A1M8: dùng để quét môi trường xung quanh, phát hiện vật cản và hỗ trợ dẫn đường
- Encoder: dùng để đo tốc độ quay và quãng đường di chuyển của bánh xe, phục vụ cho việc điều khiển chính xác vận tốc và vị trí

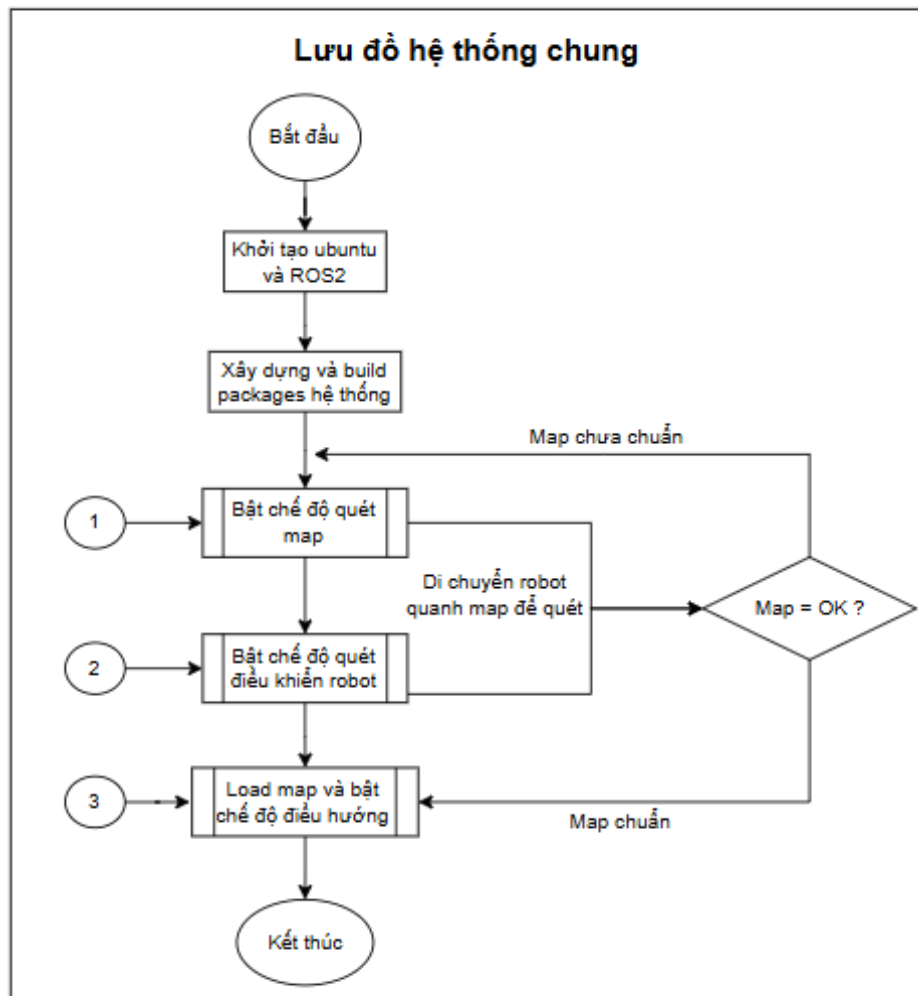
-Cơ chế để vật phẩm

Cơ chế để thuốc được thiết kế gồm 2 tầng chứa vật phẩm, nhằm:

- Tăng dung lượng lưu trữ
- Phân loại vật phẩm dễ dàng

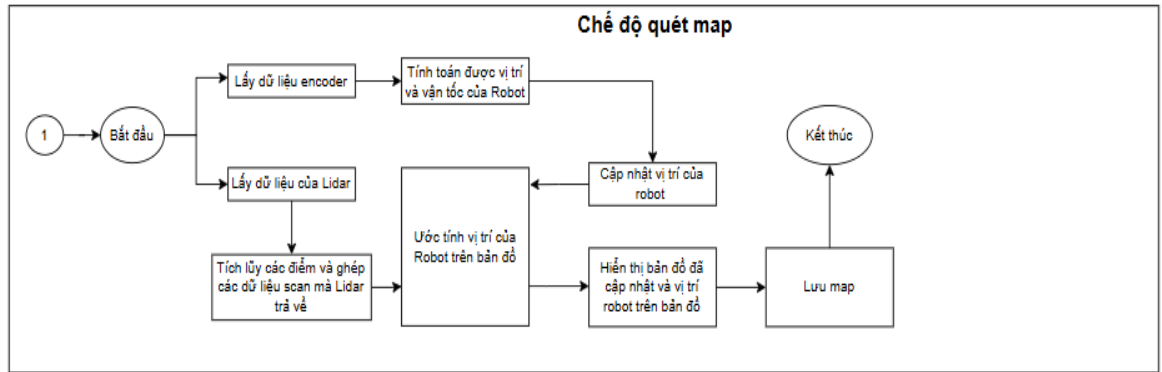
Thiết kế dạng nhiều tầng giúp tận dụng tối đa không gian theo phương thẳng đứng của thân xe.

3.5. 2 Lưu đồ giải thuật tổng quát hệ thống chung



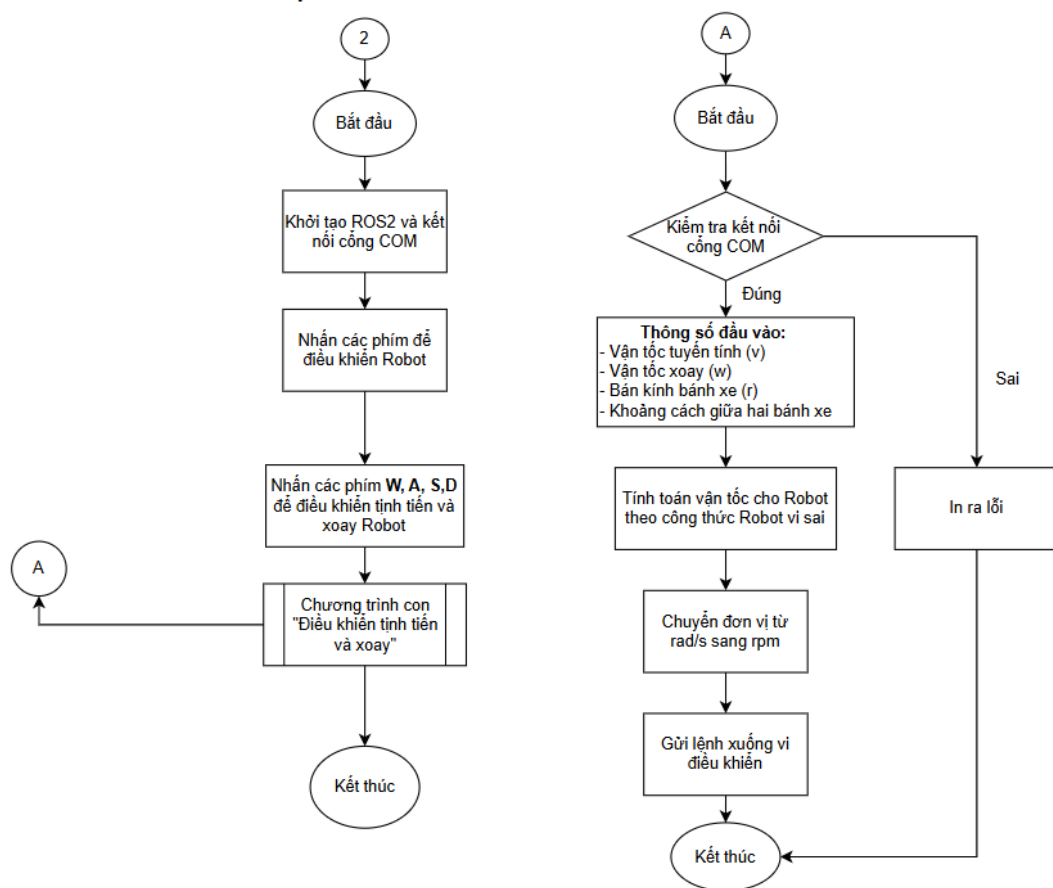
Hình 3. 20 Lưu đồ giải thuật hệ thống chung

Lưu đồ hệ thống chung mô tả quy trình vận hành tổng thể của robot trong môi trường ROS2. Hệ thống được khởi tạo từ Ubuntu và ROS2, sau đó tiến hành xây dựng và biên dịch các package cần thiết. Robot hoạt động ở chế độ quét bản đồ bằng cách di chuyển trong môi trường để thu thập dữ liệu. Khi bản đồ chưa đạt yêu cầu, quá trình quét và điều khiển robot tiếp tục lặp lại. Sau khi bản đồ đạt độ chính xác cần thiết, hệ thống chuyển sang chế độ nạp bản đồ và điều hướng, kết thúc quá trình vận hành.



Hình 3. 21 sơ đồ chế độ quét map

Lưu đồ chế độ quét bản đồ mô tả quá trình xây dựng bản đồ môi trường của robot. Robot thu thập dữ liệu từ encoder để tính toán vận tốc và vị trí, đồng thời lấy dữ liệu từ cảm biến LiDAR để quét môi trường xung quanh. Các dữ liệu quét được tích lũy và ghép lại nhằm ước tính vị trí robot trên bản đồ và cập nhật bản đồ theo thời gian thực. Vị trí và bản đồ sau khi cập nhật được hiển thị trực quan, cuối cùng bản đồ hoàn chỉnh được lưu lại để phục vụ cho quá trình điều hướng.



Hình 3. 22 Lưu đồ giải thuật chế độ quét và điều khiển robot

Lưu đồ mô tả quy trình điều khiển robot ở chế độ thủ công thông qua bàn phím và giao tiếp với vi điều khiển qua cổng COM. Hệ thống gồm hai chức năng chính: (1) điều khiển robot di chuyển theo vận tốc tịnh tiến và vận tốc quay; (2) cập nhật/điều chỉnh tốc độ theo hệ số scale trong quá trình vận hành.

Khởi “Chế độ điều khiển robot”

Quá trình bắt đầu bằng việc khởi tạo ROS2 và thiết lập kết nối cổng COM để đảm bảo máy tính (ROS2) có thể gửi lệnh điều khiển xuống vi điều khiển. Sau khi kết nối thành công, hệ thống chuyển sang trạng thái lắng nghe phím điều khiển từ người dùng.

Tại đây, bàn phím được chia làm 2 nhóm lệnh:

Nhóm điều khiển chuyển động (W, A, S, D)

Người dùng nhấn các phím điều hướng để tạo lệnh chuyển động cơ bản:

- W/S: tăng/giảm vận tốc tịnh tiến hoặc tiến/lùi

- X là dừng chuyển động
 - A/D: thay đổi hướng quay trái/phải (vận tốc góc)
- Các lệnh này được chuyển sang chương trình con (A) “Điều khiển tịnh tiến và xoay” để tính toán vận tốc bánh xe và gửi xuống vi điều khiển.

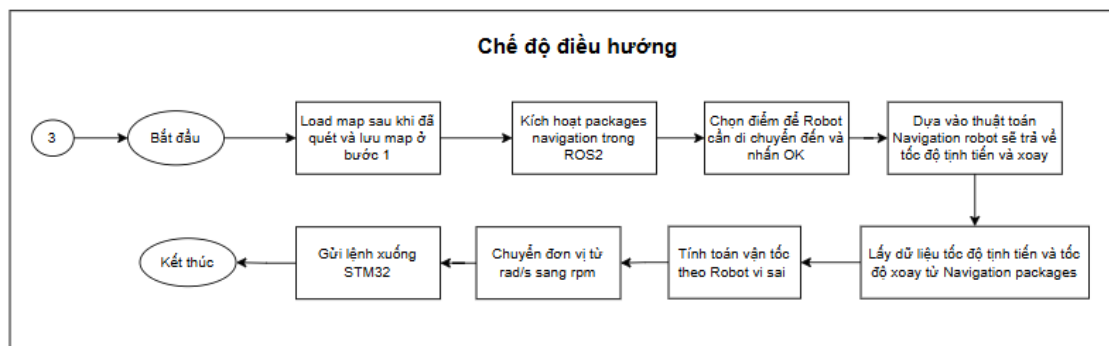
Chương trình con (A) “Điều khiển tịnh tiến và xoay”

Ở nhánh (A), hệ thống trước hết kiểm tra trạng thái kết nối COM:

- Nếu không kết nối → hệ thống in ra lỗi và dừng nhánh xử lý.
- Nếu kết nối đúng → hệ thống nhận các tham số đầu vào gồm:
 - o Vận tốc tịnh tiến v
 - o Vận tốc quay w
 - o Bán kính bánh xe r
 - o Khoảng cách giữa hai bánh xe (thông số hình học)

Sau đó, hệ thống tính toán vận tốc cho robot theo mô hình động học (robot vi sai/differential drive), chuyển từ vận tốc tịnh tiến–quay sang vận tốc góc của từng bánh (bánh trái/bánh phải). Tiếp theo, các giá trị vận tốc được chuyển đổi đơn vị từ rad/s sang rpm để phù hợp với chuẩn điều khiển động cơ trên vi điều khiển.

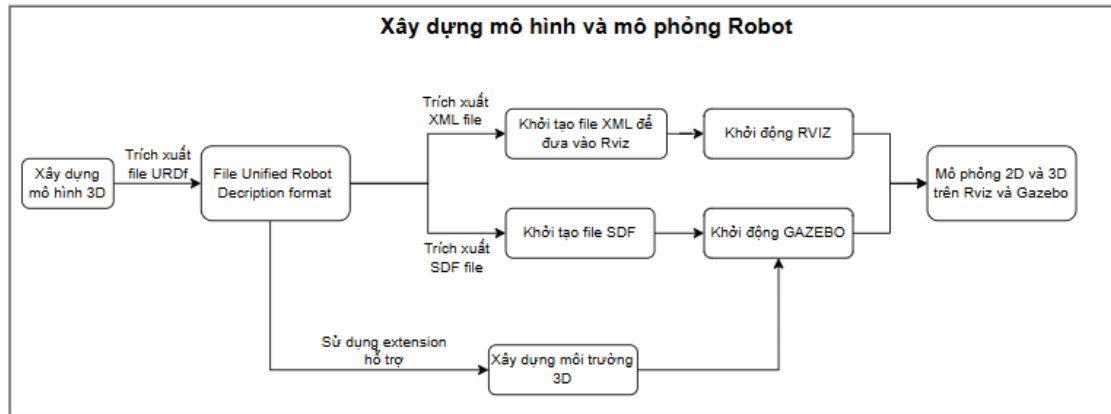
Cuối cùng, lệnh rpm được đóng gói và gửi xuống VĐK, VĐK sẽ điều khiển driver động cơ để robot thực thi chuyển động tương ứng.



Hình 3. 23 sơ đồ chi tiết các bước ở chế độ điều hướng

Lưu đồ chế độ điều hướng mô tả quá trình robot di chuyển tự động dựa trên bản đồ đã xây dựng. Hệ thống nạp bản đồ thu được từ bước quét map, sau đó kích hoạt các package Navigation trong ROS2 và lựa chọn điểm đích cần di chuyển. Thuật toán điều hướng tính toán quỹ đạo, vận tốc tịnh tiến và vận tốc quay của robot, đồng thời sử dụng dữ liệu này để xác định vận tốc riêng cho từng bánh xe. Các giá trị vận tốc được chuyển đổi từ rad/s sang rpm và gửi xuống vi điều khiển để thực hiện chuyển động, kết thúc quá trình điều hướng.

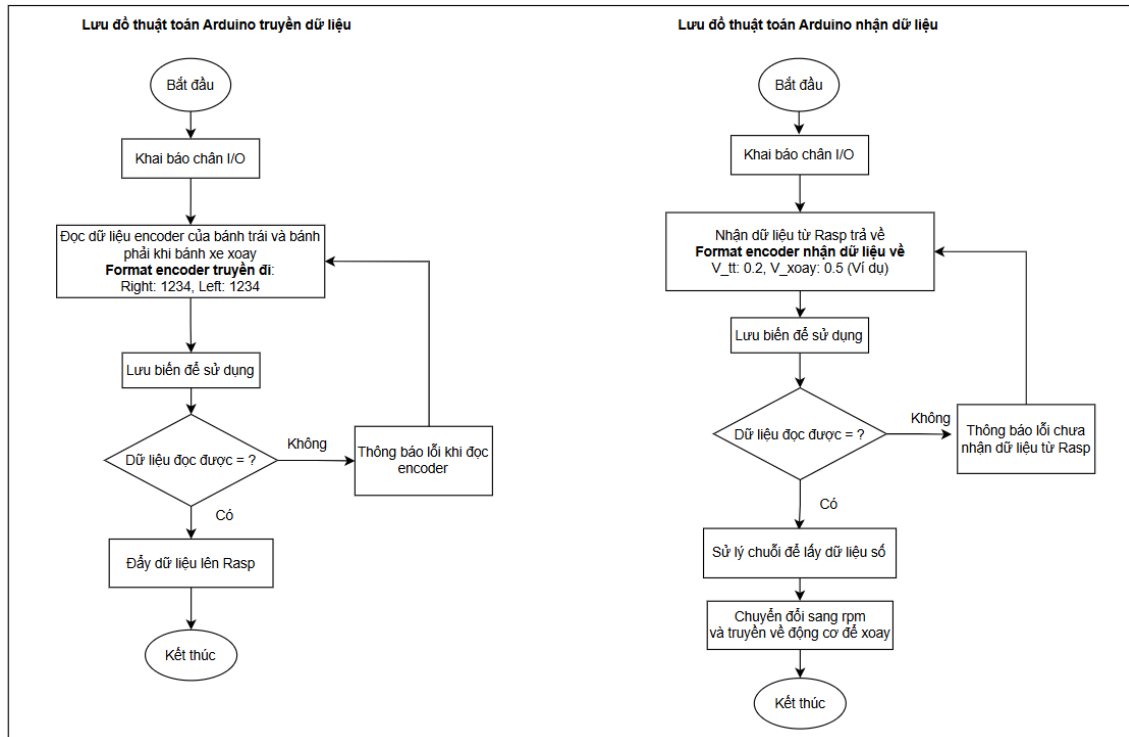
3.5.3 Sơ đồ xây dựng mô hình và mô phỏng robot



Hình 3. 24 sơ đồ xây dựng mô hình và mô phỏng Robot

Lưu đồ mô tả quy trình xây dựng mô hình và mô phỏng robot trong ROS. Mô hình 3D của robot được thiết kế và trích xuất sang định dạng URDF, đóng vai trò trung tâm mô tả cấu trúc robot. Từ URDF, dữ liệu được chuyển đổi sang file XML để trực quan hóa trên RViz và sang file SDF để mô phỏng vật lý trong Gazebo. Đồng thời, môi trường 3D được xây dựng và tích hợp vào Gazebo. Quá trình này cho phép mô phỏng và quan sát hoạt động của robot trên cả RViz và Gazebo trước khi triển khai thực tế.

3.5.4. Lưu đồ giải thuật xử lý encoder trên arduino mega2560



Hình 3. 25 lưu đồ thuật toán Arduino mega truyền và nhận dữ liệu

Ở nhánh Arduino truyền dữ liệu, hệ thống trước hết thực hiện khởi tạo và cấu hình các chân I/O cần thiết. Sau đó, Arduino liên tục đọc giá trị encoder của bánh trái và bánh phải để xác định số xung quay của từng bánh xe. Dữ liệu encoder sau khi thu thập sẽ được lưu vào các biến tạm để xử lý.

Tiếp theo, hệ thống kiểm tra tính hợp lệ của dữ liệu encoder:

- Nếu dữ liệu không đọc được hoặc xảy ra lỗi trong quá trình thu thập, Arduino sẽ thông báo lỗi và tiếp tục thực hiện chu trình đọc dữ liệu mới.
- Nếu dữ liệu hợp lệ, giá trị encoder của hai bánh xe sẽ được đóng gói theo định dạng quy định và truyền lên Raspberry Pi thông qua giao tiếp Serial.

Ở nhánh Arduino nhận dữ liệu, hệ thống cũng bắt đầu bằng việc khởi tạo các chân I/O phục vụ điều khiển động cơ. Sau đó, Arduino liên tục lắng nghe và nhận dữ liệu điều khiển được gửi từ Raspberry Pi.

Khi nhận được dữ liệu:

- Nếu không nhận được dữ liệu hoặc dữ liệu không hợp lệ, hệ thống sẽ thông báo lỗi và tiếp tục chờ dữ liệu mới.
- Nếu dữ liệu hợp lệ, Arduino tiến hành xử lý chuỗi nhận được để tách các giá trị điều khiển tương ứng cho từng bánh xe.

Cuối cùng, các giá trị điều khiển được chuyển đổi thành tín hiệu PWM và xuất đến driver động cơ. Driver sẽ điều khiển động cơ quay theo tốc độ và chiều quay tương ứng, giúp robot thực hiện các chuyển động như tiến, lùi hoặc quay theo yêu cầu từ Raspberry Pi.

3.6 Thiết kế website

3.6.1 Giao diện và chức năng của website quản lý

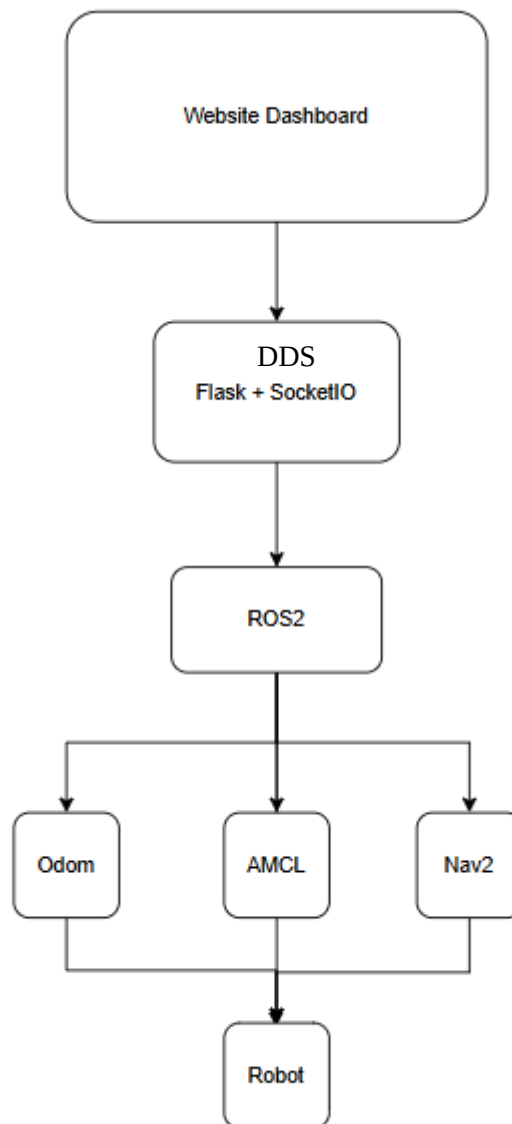
Giao diện giúp người quản lý kết nối được với Robot, thiết lập được vị trí các điểm đến trên bản đồ và chỉ định robot di chuyển đến điểm đến nhằm mục đích phục vụ yêu cầu của khách hàng. Bên cạnh đó, website còn giúp theo dõi robot theo thời gian thực trên bản đồ và xác định đã đến vị trí chỉ định hay chưa.

3.6.2 Kết nối website và robot

Trong hệ thống đề tài, website được xây dựng trên nền tảng Flask và kết nối truyền thông DDS với hệ thống ROS2 đang vận hành trên Raspberry Pi 4. Thông qua thư viện rclpy, website có thể truy cập các topic và action của ROS2 để trao đổi dữ liệu với robot.

Để đảm bảo dữ liệu được cập nhật theo thời gian thực, hệ thống sử dụng Flask-SocketIO làm cầu nối giữa giao diện web và ROS2. Các thông tin như vị trí robot, dữ liệu odometry và trạng thái điều hướng được ROS2 gửi lên website để hiển thị cho người vận hành. Ngược lại, các lệnh điều khiển từ website sẽ được chuyển thành các mục tiêu điều hướng đến Navigation2 thông qua action NavigateToPose.

Nhờ cơ chế này, website có thể theo dõi trạng thái hoạt động của robot, quản lý các vị trí phục vụ và thực hiện điều hướng từ xa thông qua một giao diện trực quan, góp phần nâng cao khả năng quản lý và vận hành hệ thống.



Hình 3. 26 Sơ đồ mô tả luồng dữ liệu giao tiếp giữa Web với ROS

KẾT THÚC CHƯƠNG 3

Qua chương 3, nhóm đã hoàn thành việc thiết kế và xây dựng hệ thống robot phục vụ nhà hàng tự hành dựa trên nền tảng ROS2. Các thành phần phần cứng và phần mềm đã được tích hợp đồng bộ, cho phép robot thực hiện chức năng xây dựng bản đồ, định vị và điều hướng tự động trong môi trường thực tế. Kết quả đạt được là một hệ thống

robot có khả năng vận hành ổn định, đáp ứng các yêu cầu đề ra và sẵn sàng cho quá trình thực nghiệm, đánh giá ở chương tiếp theo.

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

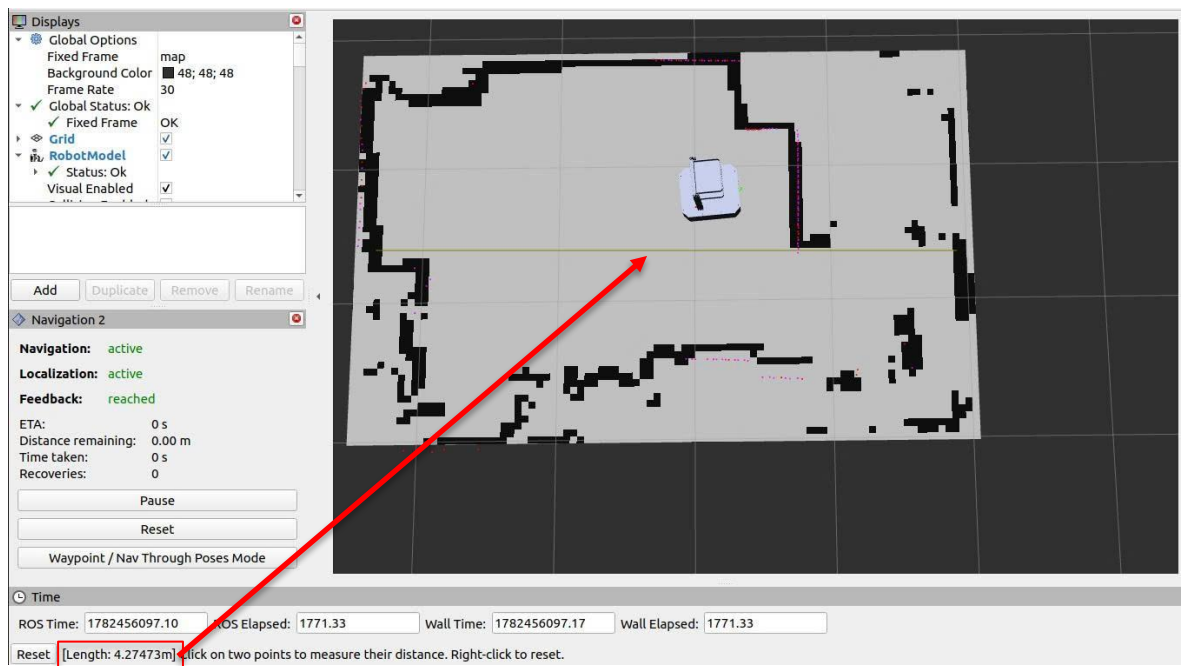
Chương 4 trình bày quá trình thực nghiệm và đánh giá hệ thống robot phục vụ nhà hàng tự hành sau khi hoàn thiện thiết kế và xây dựng. Các thử nghiệm được thực hiện trong môi trường thực tế nhằm kiểm tra khả năng xây dựng bản đồ, định vị, điều hướng và tránh vật cản của robot. Đồng thời, kết quả hoạt động của robot trên môi trường thực và môi trường mô phỏng RViz2 cũng được so sánh và đánh giá. Thông qua các thí nghiệm này, mức độ đáp ứng của hệ thống đối với các yêu cầu đặt ra sẽ được xác định.

4.1 Môi trường thực nghiệm thực tế và Rviz2

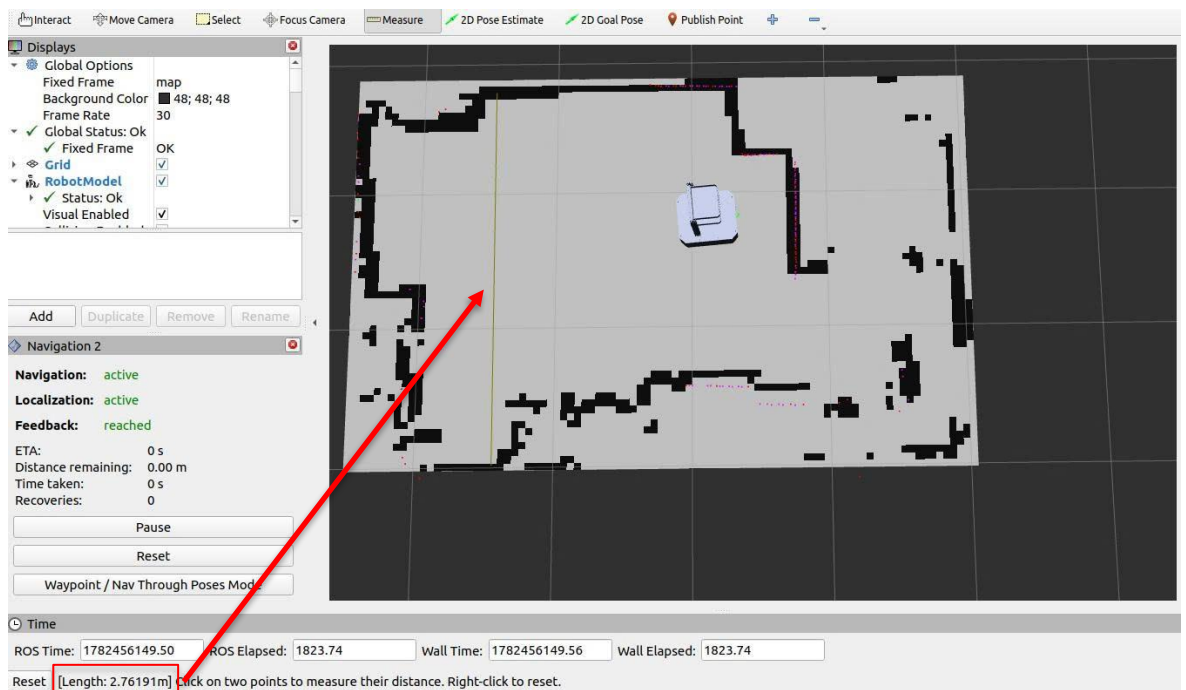
Các thử nghiệm được thực hiện trong môi trường trong nhà nhằm mô phỏng điều kiện hoạt động của robot phục vụ nhà hàng. Khu vực thực nghiệm có kích thước khoảng $4,27\text{ m} \times 2,76\text{ m}$, bao gồm các vật cản cố định như tường, bàn và các khu vực di chuyển của robot. Môi trường được bố trí phù hợp để đánh giá khả năng xây dựng bản đồ, định vị và điều hướng tự động của hệ thống.

Trong giai đoạn xây dựng bản đồ, robot được điều khiển di chuyển trong toàn bộ khu vực thực nghiệm để thu thập dữ liệu từ cảm biến LiDAR. Kết quả thu được là bản đồ 2D của môi trường được hiển thị trên RViz2 sai số không quá 1% với kích thước khoảng $4,2747 \times 2,7619\text{ m}$. Trên giao diện RViz2, mỗi ô lưới lớn tương ứng với chiều dài 1 m trong môi trường thực tế, cho phép dễ dàng đối chiếu và đánh giá độ chính xác của bản đồ.

Môi trường thực nghiệm này được sử dụng xuyên suốt quá trình kiểm tra các chức năng của hệ thống, bao gồm xây dựng bản đồ bằng SLAM, định vị bằng AMCL, điều hướng bằng Navigation2 và thực hiện các nhiệm vụ phục vụ trong mô hình nhà hàng.



Hình 4. 1 Kết quả đo chiều dài bản đồ trên Rviz2



Hình 4. 2 Kết quả đo chiều rộng trên Rviz2



Hình 4. 3 Diện tích môi trường thực tế

Bảng 4. 1 Kết quả kiểm thử môi trường thực tế so với

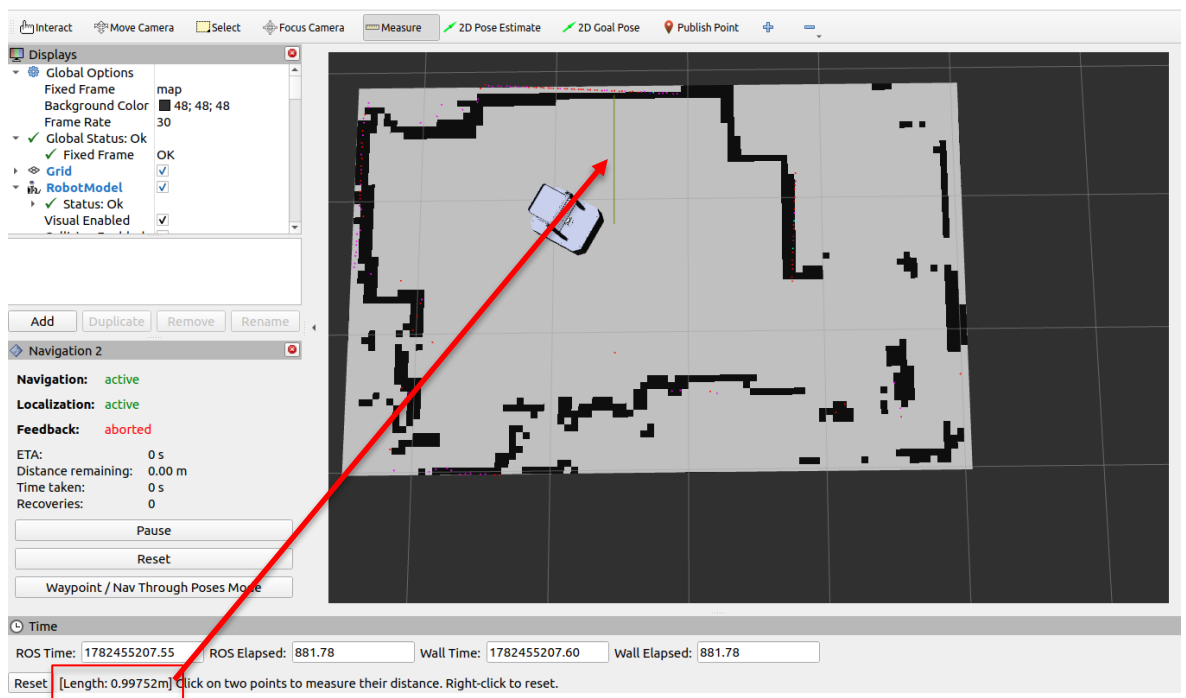
Hạng mục	Nội dung
Diện tích trên thực tế	4.27x2.76
Diện tích trên Rviz2	$4,2747 \times 2,7619 \text{ m}$
Tỉ lệ sai số chiều dài	$> 1\%$
Tỉ lệ sai số chiều rộng	$> 1\%$
Phương pháp đo trên Rviz2	Measure
Phương pháp đo trên môi trường	Thước đo 5m

4.2 Thực nghiệm vị trí Robot trong thực tế và trong Rviz2

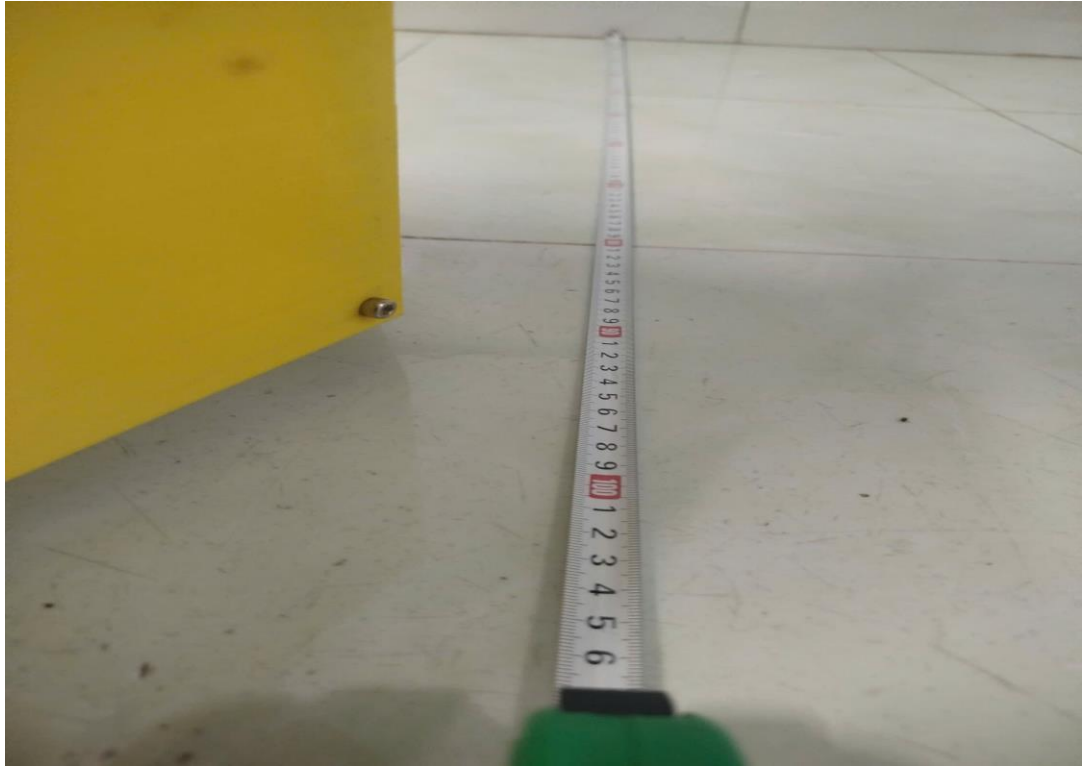
Trong quá trình thực nghiệm, robot được đặt tại nhiều vị trí khác nhau trong khu vực hoạt động. Tại mỗi vị trí, tọa độ và hướng của robot trong thực tế được đối chiếu với vị trí hiển thị trên RViz2 thông qua khung tọa độ của hệ thống ROS2. Kết quả quan sát cho thấy vị trí của robot trên bản đồ số trùng khớp với vị trí thực tế trong

môi trường với sai số nhỏ hơn 1%. Khi robot di chuyển theo các tuyến đường thẳng, thực hiện quay trái, quay phải hoặc thay đổi vị trí đột ngột, mô hình robot trên RViz2 đều cập nhật gần như tức thời và phản ánh chính xác trạng thái thực của robot.

Hình 4.4 là khoảng cách vị trí robot đo được trên Rviz2, ta có thể quan sát được kích thước từ tường đến tâm robot khoảng 0.997m. Còn hình 4.5 là khoảng cách đo ở ngoài thực tế khoảng 0.99m. Dựa vào số liệu ta có thể đánh giá được tỉ lệ sai số của hệ thống khá thấp, dưới 1%



Hình 4. 4 vị trí quan sát được trên Rviz

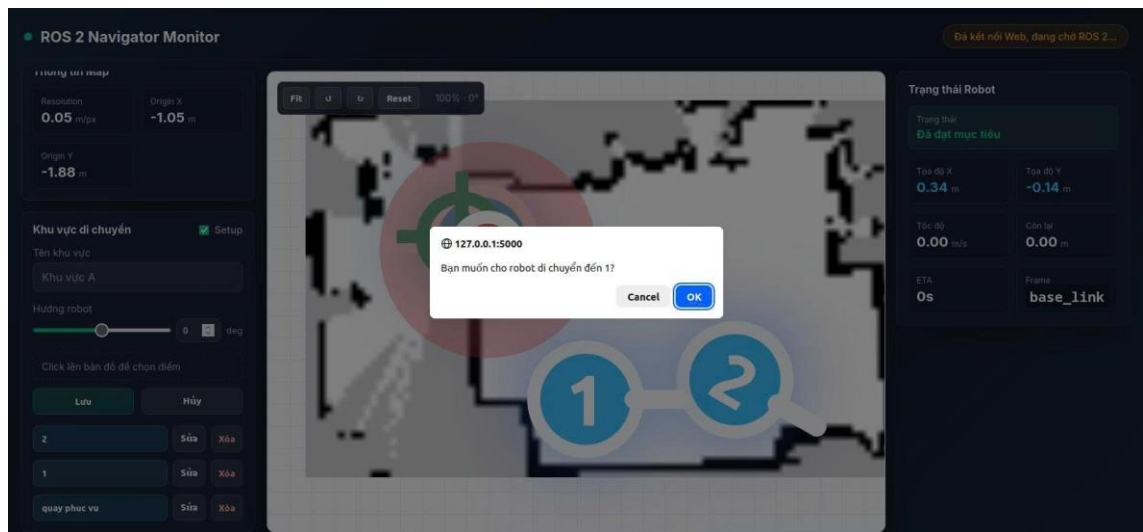


Hình 4. 5 vị trí robot thực tế bên ngoài

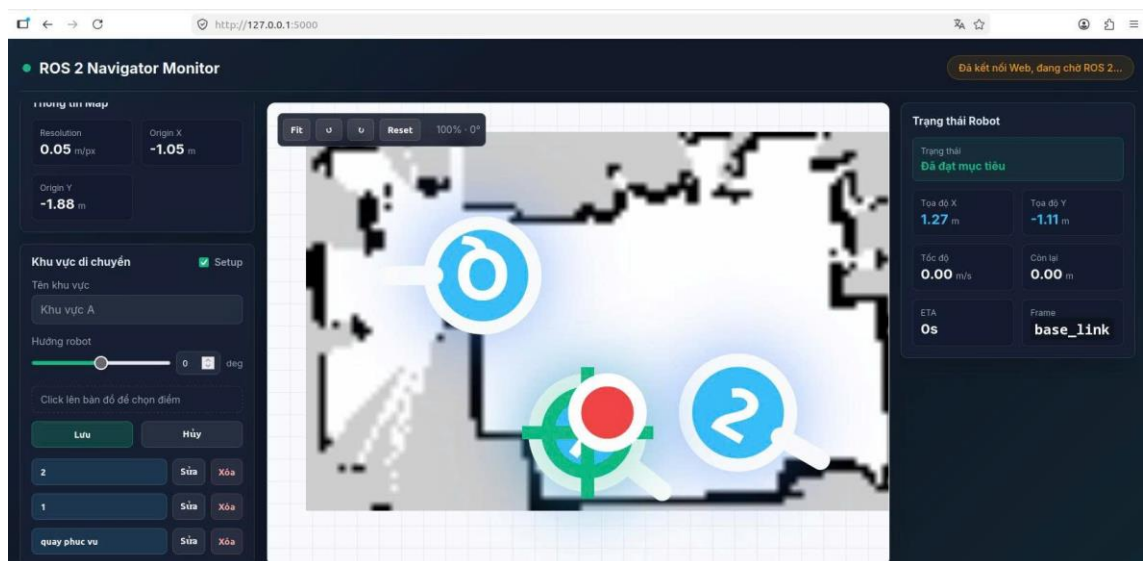
4.3 Thực nghiệm mô hình robot phục vụ nhà hàng

Trước khi thực hiện thử nghiệm các trường hợp thì chúng ta phải vào Rviz2 dùng 2D Estimate để xác định vị trí ban đầu và hướng đi của robot, sau đó dùng 2D Goal Pose để xác định vị đến và hướng đến của robot. Sau khi Robot đã chạy đến điểm cần đến trên bản đồ Rviz2 cũng như ngoài thực tế, thì ta vào website để đặt tên, chọn lại điểm đến, hướng đến của Robot trong bản đồ của website và lưu lại. Tiếp tục như thế cho các vị trí khác và ta có 3 vị trí điểm đến để kiểm nghiệm cho 3 trường hợp bên dưới.

4.3.1 Trường hợp 1: Robot đến bàn số 1 khi không có vật cản



Hình 4. 6 Giao diện xác nhận di chuyển để vị trí số 1 trước khi robot thực hiện điều hướng



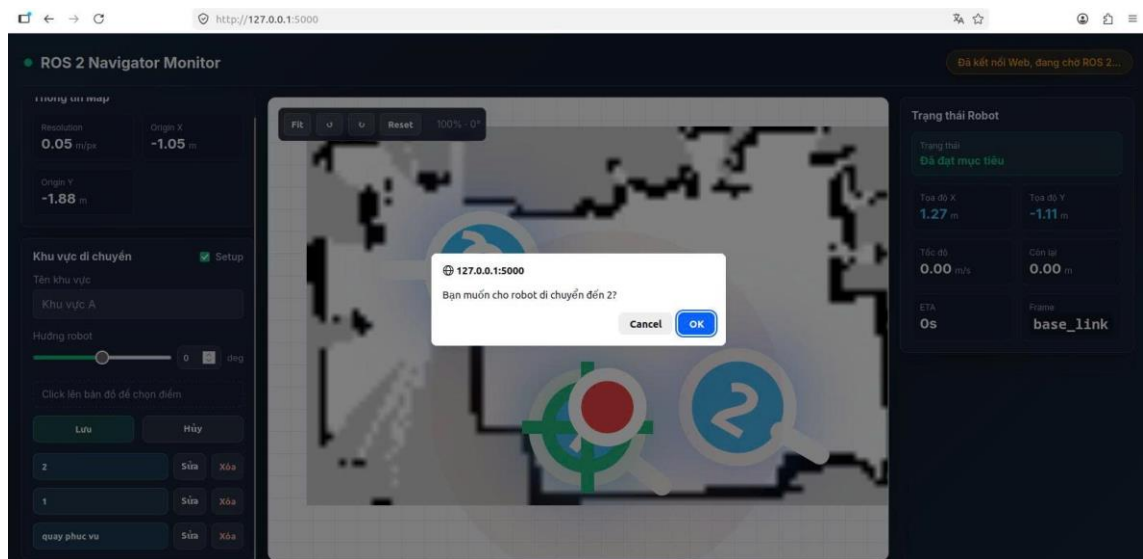
Hình 4. 7 Giao diện Robot di chuyển thành công đến vị trí mục tiêu được lựa chọn trên bản đồ



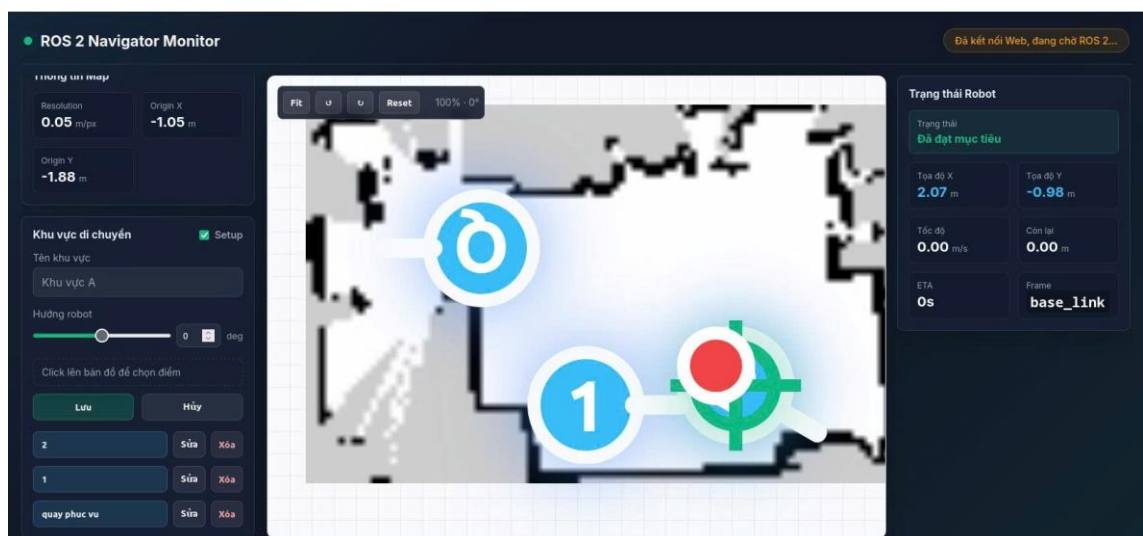
Hình 4. 8 Robot đã di chuyển đến vị trí bàn số 1 trong thực tế

Sau khi chạy thử nghiệm trường hợp 1 cho robot di chuyển đến vị trí số 1 trong môi trường thực nghiệm đã thu được kết quả như mong đợi, robot đã di chuyển đúng vị trí mong muốn được đề ra cả trên hệ thống và ngoài thực tiễn.

4.3.2 Trường hợp 2: Robot đến bàn số 2 khi không có vật cản



Hình 4. 9 Giao diện xác nhận di chuyển đến vị trí số 1 trước khi robot thực hiện điều hướng



Hình 4. 10 Giao diện Robot di chuyển thành công đến vị trí mục tiêu được lựa chọn trên bản đồ



Hình 4. 11 Robot đã di chuyển đến vị trí bàn số 2 trong thực tế

Kết quả thực nghiệm ở trường hợp 2 cho thấy hệ thống điều hướng hoạt động ổn định khi thực hiện nhiệm vụ di chuyển đến vị trí mục tiêu số 2. Sau khi nhận lệnh điều hướng, robot đã lập kế hoạch đường đi và di chuyển thành công đến vị trí đã được thiết lập trước. Quan sát trên giao diện giám sát cho thấy trạng thái robot được cập nhật chính xác, đồng thời vị trí hiển thị trên bản đồ phù hợp với vị trí thực tế của robot trong môi trường thực nghiệm. Điều này chứng minh khả năng thực hiện nhiệm vụ điều hướng tự động của hệ thống với độ chính xác và độ tin cậy cao.

4.3.3 Trường hợp 3: Robot quay về quầy phục vụ



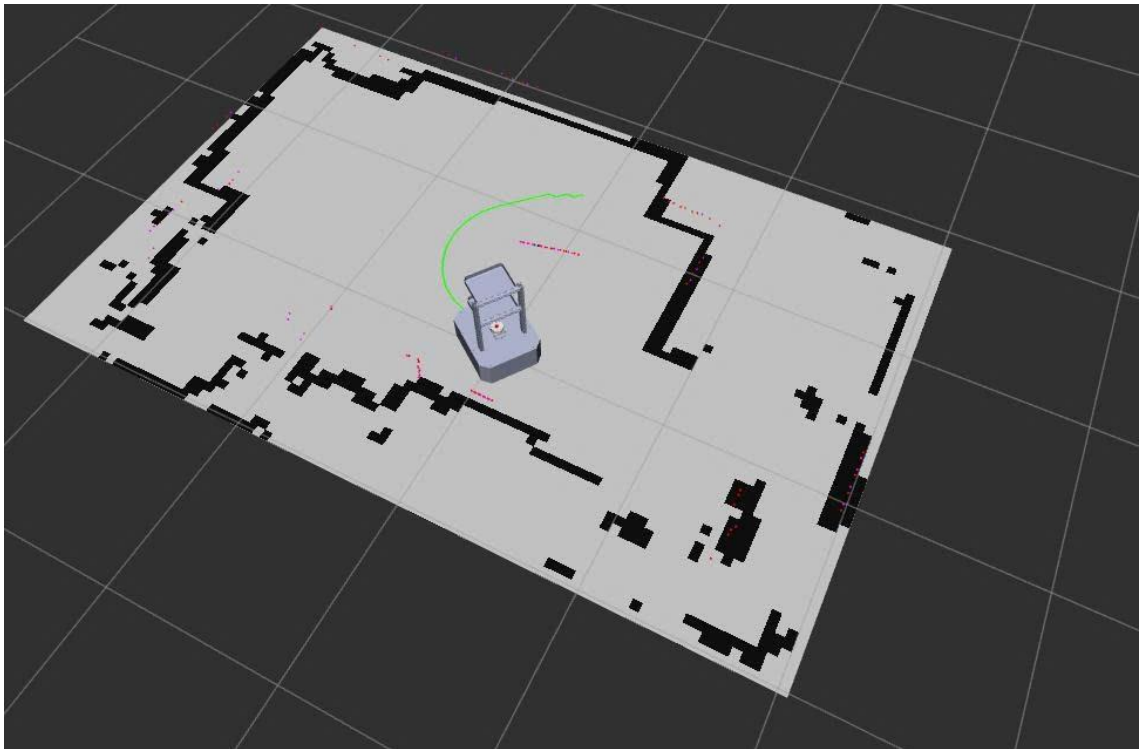
Hình 4. 12 Giao diện Robot di chuyển thành công quầy phục vụ trên bản đồ



Hình 4. 13 Robot đã đến quầy ngoài thực tế

Kết quả thực nghiệm ở trường hợp 3 cho thấy robot đã thực hiện thành công nhiệm vụ quay trở về quầy phục vụ sau khi nhận lệnh từ hệ thống. Trong quá trình di chuyển, robot sử dụng dữ liệu định vị và thuật toán điều hướng để xác định lộ trình phù hợp, đồng thời tránh va chạm với các vật cản trong môi trường. Sau khi hoàn thành quá trình điều hướng, robot đã dừng đúng tại vị trí quầy phục vụ được thiết lập trước. Kết quả quan sát trên giao diện giám sát và trong môi trường thực tế cho thấy vị trí của robot hoàn toàn phù hợp với mục tiêu đề ra. Điều này chứng minh hệ thống có khả năng thực hiện các nhiệm vụ phục vụ lặp lại một cách chính xác, ổn định và đáp ứng tốt yêu cầu vận hành trong môi trường nhà hàng.

4.3.4 Trường hợp 4: Robot di chuyển và tránh né khi có vật cản phía trước



Hình 4. 14 Robot xử lý chuyển hướng lựa chọn đường đi đi né vật cản trên Rviz



Hình 4. 15 Robot chuyển hướng ngoài thực tế

Sau khi nhận lệnh di chuyển đến vị trí số 1, robot bắt đầu thực hiện điều hướng theo lộ trình được Nav2 lập kế hoạch. Khi vật cản xuất hiện trong vùng quét của cảm biến LiDAR, hệ thống đã phát hiện và cập nhật thông tin vào bản đồ chi phí cục bộ (Local Costmap). Dựa trên dữ liệu cảm biến thu được, bộ điều hướng tự động tính toán lại quỹ đạo di chuyển phù hợp để tránh va chạm với vật cản đồng thời vẫn đảm bảo hướng tới mục tiêu đã được thiết lập.

4.4 Nhận xét và đánh giá

4.4.1 Đánh giá kết quả đạt được

Qua các nội dung thực nghiệm đã thực hiện, hệ thống robot phục vụ nhà hàng đã đáp ứng được các mục tiêu đề ra trong phạm vi nghiên cứu. Robot có khả năng xây dựng bản đồ môi trường trong nhà, xác định vị trí hiện tại trên bản đồ và thực hiện điều hướng tự động đến các vị trí mục tiêu với độ chính xác cao. Kết quả thực nghiệm

cho thấy vị trí của robot trong môi trường thực tế tương ứng tốt với vị trí hiển thị trên RViz2 và giao diện giám sát, chứng minh hiệu quả của quá trình tích hợp giữa dữ liệu odometry, cảm biến LiDAR và thuật toán định vị AMCL.

Các chức năng điều hướng được triển khai trên nền tảng Nav2 hoạt động ổn định trong môi trường thực nghiệm. Robot có thể di chuyển đến các vị trí được lựa chọn trên bản đồ, thực hiện quay trở về quầy phục vụ và hoàn thành các nhiệm vụ đã được thiết lập trước. Trong các lần thử nghiệm, robot đều có khả năng lập kế hoạch đường đi phù hợp và dừng đúng tại vị trí mục tiêu với sai số nhỏ.

Bên cạnh đó, hệ thống cũng thể hiện khả năng thích ứng với sự thay đổi của môi trường thông qua chức năng tránh vật cản. Khi xuất hiện vật cản trên lộ trình di chuyển, robot có thể phát hiện, cập nhật thông tin vào bản đồ chi phí cục bộ và tự động tính toán lại quỹ đạo để tiếp tục di chuyển đến đích mà không xảy ra va chạm. Điều này cho thấy giải pháp điều hướng được xây dựng có tính an toàn và độ tin cậy cao trong môi trường hoạt động thực tế.

Giao diện Web Monitor được xây dựng đã hỗ trợ hiệu quả cho việc giám sát và tương tác với robot. Người vận hành có thể theo dõi vị trí, trạng thái hoạt động của robot theo thời gian thực, lựa chọn vị trí đích trực tiếp trên bản đồ và gửi các lệnh điều khiển từ xa một cách thuận tiện. Việc kết hợp giữa hệ thống robot và nền tảng web giúp nâng cao tính trực quan và khả năng quản lý trong quá trình vận hành.

Tuy nhiên, hệ thống vẫn còn một số hạn chế nhất định. Quá trình định vị có thể bị ảnh hưởng khi môi trường thay đổi lớn so với bản đồ đã xây dựng hoặc khi xuất hiện nhiều vật cản động cùng lúc. Ngoài ra, phạm vi thử nghiệm mới được thực hiện trong môi trường trong nhà với diện tích tương đối nhỏ nên chưa đánh giá được đầy đủ hiệu quả hoạt động trong các không gian rộng và phức tạp hơn.

Nhìn chung, kết quả thực nghiệm cho thấy hệ thống robot phục vụ nhà hàng đã hoạt động ổn định, đáp ứng tốt các yêu cầu về định vị, điều hướng, tránh vật cản và giám sát từ xa. Đây là cơ sở quan trọng để tiếp tục nghiên cứu và phát triển các chức

năng nâng cao trong tương lai nhằm tăng tính ứng dụng của robot trong các môi trường phục vụ thực tế.

4.4.2 Ưu điểm

Hệ thống có khả năng định vị và điều hướng tự động với độ chính xác cao trong môi trường trong nhà nhờ sự kết hợp giữa cảm biến LiDAR, dữ liệu odometry và thuật toán định vị AMCL.

Robot có khả năng lập kế hoạch đường đi và di chuyển đến các vị trí mục tiêu một cách tự động thông qua bộ điều hướng Nav2, giúp giảm sự can thiệp của con người trong quá trình vận hành.

Khả năng phát hiện và tránh vật cản hoạt động hiệu quả. Khi xuất hiện chướng ngại vật trên lộ trình, robot có thể tự động tính toán lại quỹ đạo di chuyển để tiếp tục đến đích mà không xảy ra va chạm.

Giao diện Web Monitor được xây dựng trực quan, cho phép người dùng dễ dàng theo dõi trạng thái hoạt động, vị trí hiện tại của robot và gửi lệnh điều khiển từ xa thông qua mạng nội bộ.

Hệ thống hỗ trợ lựa chọn trực tiếp vị trí đích trên bản đồ, giúp quá trình điều khiển robot trở nên thuận tiện và dễ sử dụng hơn đối với người vận hành.

Kiến trúc ROS2 mang tính mô-đun cao, thuận lợi cho việc mở rộng và phát triển thêm các chức năng mới trong tương lai như nhận diện bàn ăn, quản lý đơn hàng hoặc điều phối nhiều robot cùng hoạt động.

Chi phí triển khai tương đối thấp do sử dụng các phần cứng phổ biến như Raspberry Pi, Arduino, LiDAR và các động cơ DC, phù hợp cho mục đích nghiên cứu và ứng dụng thực tế quy mô nhỏ.

Hệ thống hoạt động ổn định trong các thử nghiệm thực tế, đáp ứng tốt các yêu cầu cơ bản của một robot phục vụ trong môi trường nhà hàng.

4.4.3 Hạn chế

- Hệ thống hiện chỉ được thử nghiệm trong môi trường trong nhà có diện tích tương đối nhỏ, do đó chưa đánh giá được đầy đủ hiệu quả hoạt động trong các không gian rộng hoặc có cấu trúc phức tạp hơn.
- Độ chính xác của quá trình định vị phụ thuộc vào chất lượng bản đồ đã xây dựng. Khi môi trường thay đổi đáng kể so với bản đồ ban đầu, khả năng định vị của robot có thể bị ảnh hưởng.
- Robot chủ yếu tránh được các vật cản tĩnh hoặc vật cản di chuyển với tốc độ thấp. Đối với môi trường có nhiều người qua lại hoặc vật cản động xuất hiện liên tục, hiệu quả điều hướng có thể giảm.
- Hệ thống chưa tích hợp các chức năng nhận diện đối tượng bằng camera và trí tuệ nhân tạo, do đó robot chưa thể phân biệt người, bàn ăn hoặc các loại vật thể khác nhau trong môi trường hoạt động.
- Tốc độ di chuyển của robot còn được giới hạn nhằm đảm bảo độ ổn định của quá trình định vị và tránh vật cản, dẫn đến thời gian thực hiện nhiệm vụ chưa thực sự tối ưu.
- Giao diện Web Monitor hiện chủ yếu phục vụ cho việc giám sát và điều khiển cơ bản. Các chức năng quản lý đơn hàng, quản lý bàn ăn hoặc thống kê dữ liệu vận hành vẫn chưa được triển khai.
- Hệ thống chưa hỗ trợ khả năng phối hợp nhiều robot cùng hoạt động trong một môi trường, do đó chưa đáp ứng được các bài toán phục vụ quy mô lớn.
- Thời lượng hoạt động của robot phụ thuộc vào dung lượng pin và chưa được tích hợp chức năng tự động quay về trạm sạc khi mức năng lượng thấp.
- Sai số odometry có thể tích lũy trong quá trình di chuyển quãng đường dài, đặc biệt khi bánh xe bị trượt hoặc môi trường có bề mặt không đồng đều.

KẾT THÚC CHƯƠNG 4

Kết quả thực nghiệm cho thấy hệ thống robot có khả năng định vị và di chuyển đến các vị trí mục tiêu với độ chính xác cao trong môi trường nhà hàng mô phỏng. Robot hoạt động ổn định, thực hiện tốt chức năng điều hướng tự động và tránh vật cản trên đường di chuyển. Những kết quả đạt được đã chứng minh tính khả thi của giải pháp được đề xuất, đồng thời là cơ sở để đánh giá ưu điểm, hạn chế và định hướng phát triển của hệ thống trong tương lai.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong phạm vi đồ án, nhóm đã nghiên cứu, thiết kế và xây dựng thành công hệ thống robot phục vụ nhà hàng dựa trên nền tảng ROS2. Hệ thống được tích hợp các thành phần chính bao gồm Raspberry Pi, Arduino, cảm biến LiDAR, hệ thống động cơ dẫn động vi sai, thuật toán định vị AMCL, bộ điều hướng Nav2 và giao diện giám sát Web Monitor.

Quá trình thực hiện đã hoàn thành các mục tiêu đề ra như xây dựng bản đồ môi trường trong nhà, xác định vị trí robot trên bản đồ, điều hướng tự động đến các vị trí mục tiêu, tránh vật cản trong quá trình di chuyển và giám sát hoạt động của robot thông qua giao diện web. Bên cạnh đó, hệ thống cho phép người dùng lựa chọn vị trí đích trực tiếp trên bản đồ hoặc điều khiển robot thực hiện các nhiệm vụ phục vụ đã được thiết lập sẵn.

Thông qua các thử nghiệm thực tế, robot đã thể hiện khả năng hoạt động ổn định trong môi trường trong nhà với kích thước $4,4\text{ m} \times 2,8\text{ m}$. Kết quả thực nghiệm cho thấy vị trí của robot trong môi trường thực tế có sự tương đồng cao với vị trí hiển thị trên RViz2 và giao diện giám sát. Robot có thể di chuyển chính xác đến các vị trí mục tiêu, quay về quầy phục vụ theo yêu cầu và thực hiện tránh vật cản hiệu quả khi xuất hiện chướng ngại vật trên lộ trình di chuyển.

Các kết quả đạt được đã chứng minh tính khả thi của việc ứng dụng ROS2, Nav2 và công nghệ định vị bằng LiDAR trong việc xây dựng hệ thống robot phục vụ tự hành. Đồng thời, đồ án cũng góp phần tạo nền tảng cho việc nghiên cứu và phát triển các hệ thống robot dịch vụ thông minh có khả năng ứng dụng trong các nhà hàng, quán cà phê và các môi trường phục vụ tương tự.

Mặc dù vẫn còn một số hạn chế nhất định, hệ thống đã đáp ứng tốt các yêu cầu cơ bản của một robot phục vụ tự động, đồng thời tạo tiền đề cho các nghiên cứu mở rộng và nâng cao trong tương lai.

5.2 Hướng phát triển

Trong tương lai, hệ thống có thể được tiếp tục nghiên cứu và phát triển theo các hướng sau:

- Tích hợp camera và trí tuệ nhân tạo để nhận diện bàn ăn, khách hàng và vật cản.
- Xây dựng chức năng quản lý đơn hàng và gọi món trực tiếp trên giao diện web.
- Nâng cao khả năng tránh vật cản động trong môi trường đông người.
- Phát triển chức năng tự động quay về trạm sạc khi pin yếu.
- Mở rộng phạm vi hoạt động trong môi trường có diện tích lớn hơn.
- Xây dựng hệ thống điều phối nhiều robot phục vụ cùng lúc.
- Tối ưu thuật toán điều hướng nhằm giảm thời gian di chuyển và nâng cao hiệu suất phục vụ.
- Phát triển ứng dụng trên thiết bị di động để thuận tiện cho việc giám sát và điều khiển từ xa.

KẾT LUẬN CHUNG

Đề tài “THIẾT KẾ VÀ XÂY DỰNG ROBOT PHỤC VỤ TỰ HÀNH ỨNG DỤNG ROS2”, thiết kế và xây dựng thành công hệ thống robot phục vụ nhà hàng dựa trên nền tảng ROS2. Hệ thống được tích hợp các chức năng quan trọng như xây dựng bản đồ môi trường, định vị robot, điều hướng tự động, tránh vật cản và giám sát thông qua giao diện web. Kết quả thực nghiệm cho thấy robot hoạt động ổn định, có khả năng di chuyển chính xác đến các vị trí mục tiêu, tránh vật cản hiệu quả và đồng bộ tốt giữa môi trường thực tế với môi trường hiển thị trên RViz2. Những kết quả đạt được đã khẳng định tính khả thi của giải pháp đề xuất, đồng thời tạo tiền đề cho việc nghiên cứu và phát triển các hệ thống robot phục vụ thông minh có tính ứng dụng cao trong thực tế. đã nghiên cứu, thiết kế và xây dựng thành công hệ thống robot phục vụ nhà hàng dựa trên nền tảng ROS2. Hệ thống được tích hợp các chức năng quan trọng như xây dựng bản đồ môi trường, định vị robot, điều hướng tự động, tránh vật cản và

giám sát thông qua giao diện web. Kết quả thực nghiệm cho thấy robot hoạt động ổn định, có khả năng di chuyển chính xác đến các vị trí mục tiêu, tránh vật cản hiệu quả và đồng bộ tốt giữa môi trường thực tế với môi trường hiển thị trên RViz2. Những kết quả đạt được đã khẳng định tính khả thi của giải pháp đề xuất, đồng thời tạo tiền đề cho việc nghiên cứu và phát triển các hệ thống robot phục vụ thông minh có tính ứng dụng cao trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] KNAPP AG. (2024). *Open Shuttles: Autonomous mobile robots for flexible processes*. KNAPP. <https://www.knapp.com/en/insights/blog/open-shuttles-autonomous-mobile-robots-for-flexible-processes/>
- [2] 24 Market Reports. (2025). *Restaurant service robot market: Global outlook and forecast 2025–2032*. <https://www.24marketreports.com/machines/global-restaurant-service-robot-forecast-market>
- [3] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.
- [4] Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press
- [5] Rigoni, E. (2023). *ROS2 Nav2 – Generate a map with slam_toolbox*. The Robotics Back-End. https://roboticsbackend.com/ros2-nav2-generate-a-map-with-slam_toolbox/
- [6] Macenski, S., Martín, F., White, R., & Clavero, J. G. (2020). *The Marathon 2: A Navigation System*. arXiv. <https://arxiv.org/abs/2003.00368>
- [7] ROS Index. (2026). *Navigation2 Repository Overview*. <https://index.ros.org/r/navigation2/>
- [8] Matthias. (2018, June 19). *Nav Stack: One map for AMCL and one for move_base?* Robotics Stack Exchange. <https://robotics.stackexchange.com/questions/87407/nav-stack-one-map-for-amcl-and-one-for-move-base>
- [9] Fox, D. (2001). *KLD-sampling: Adaptive particle filters*. Advances in Neural Information Processing Systems.
- [10] Magdy. (2024). *Arduino BTS7960 DC motor driver interfacing tutorial*. DeepBlue Embedded. <https://deepbluembedded.com/arduino-bts7960-dc-motor-driver/>

- [11] Microchip Technology Inc. (n.d.). *ATmega640/1280/1281/2560/2561 datasheet*. <https://www.microchip.com>
- [12] Raspberry tips. “Raspberry Pi 3B+ vs Pi4: Which one to choose?”, <https://raspberrytips.com/raspberry-pi-3b-vs-4/>
- [13] Slamtec. (2023). *RPLIDAR A1 product specification*. Slamtec. <https://www.slamtec.com/en/Lidar/A1>
- [14] UTMEL. (2022). *XL4016 step-down voltage regulator: Pinout, datasheet and applications*. UTMEL Electronic. <https://www.utmel.com>